CHAPTER 1

Distributed word representations and Word2Vec

In Chapter **??**, we examined statistical language modeling through n-gram Markov chains and smoothed frequency distributions, demonstrating how purely discrete combinatorics collapses beneath the curse of dimensionality. In Chapter **??**, we established the mathematical machinery of continuous deep neural networks, proving how non-linear activations prevent rank collapse and how multivariable gradient descent optimizes parameters across continuous manifolds. Furthermore, in Chapter **??**, we explored temporal dynamics and gating mechanisms over time-varying hidden states.

Yet, a fundamental representation barrier governs every neural approach to natural language and source code: while human communication is articulated through discrete symbolic tokens, deep neural networks are continuous differentiable matrix operators that can neither compute distances nor evaluate gradients on raw string literals. Treating words as atomic indices via one-hot standard basis vectors incurs extreme sparsity, geometric orthogonality, and zero parameter sharing during backpropagation.

This chapter constructs the theoretical and mathematical bridge from discrete symbols to dense continuous metric spaces. Grounded in Harris and Firth's distributional hypothesis, we formulate self-supervised objective functions that organize latent spaces through context prediction. We dissect the Word2Vec suite—analyzing Continuous Bag-of-Words (CBOW) and Continuous Skip-Gram—and solve the full softmax computational bottleneck via Hierarchical Softmax binary trees and Noise Contrastive Negative Sampling (SGNS). We then address the out-of-vocabulary barrier and morphological blindness through FastText subword character n-gram compositions, analyze the emergent geometric symmetries of semantic clustering and linear vector analogies, and formalize modern subword tokenization and positional encoding pipelines that underpin contemporary foundation models.

1.1 The representation problem: why continuous embeddings are mandatory

Natural language and software engineering source code are fundamentally composed of discrete, symbolic tokens. In written English or in source files, programs and sentences present themselves as strings of alphanumeric characters: words such as `"compiler"`, `"optimizer"`, `"gradient"`, and keywords such as `"class"`, `"def"`, `"return"`.

Conversely, modern artificial neural networks are continuous, differentiable mathematical operators. An L-layer deep neural network $\mathcal{F}(\mathbf{x}; \Theta)$ computes outputs through compositions of parameterized linear projections and non-linear activation functions:

$$\mathcal{F}(\mathbf{x}; \Theta) = \sigma_L\Big(\mathbf{W}_L\, \sigma_{L-1}\big(\cdots \sigma_1(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)\cdots\big) + \mathbf{b}_L\Big), \tag{1.1}$$

where $\mathbf{W}_l \in \mathbb{R}^{d_l \times d_{l-1}}$ and $\mathbf{b}_l \in \mathbb{R}^{d_l}$. Such architectures rely fundamentally on the continuous mathematics of vector spaces $\mathbb{R}^d$, multivariable calculus, and gradient-based backpropagation. The optimization algorithm minimizes an empirical risk loss $\mathcal{L}(\Theta)$ by iteratively updating weight tensors along the negative gradient direction:

$$\Theta \leftarrow \Theta - \eta\, \nabla_\Theta \mathcal{L}(\Theta). \tag{1.2}$$

This mathematical reality creates an immediate, fundamental impasse: **discrete text strings cannot be differentiated**. The partial derivative of a scalar loss with respect to a raw string literal, $\frac{\partial \mathcal{L}}{\partial \texttt{"cat"}}$, is mathematically undefined. Discrete symbols possess no notion of infinitesimal perturbation, tangent spaces, or linear vector combinations. One cannot add `"cat"` to `"dog"`, nor compute an inner product between `"while"` and `"for"`.

To bridge this chasm between discrete symbolic communication and continuous gradient-based computation, we must establish a formal mapping from the discrete vocabulary into a continuous vector space.

Definition 1.1: *Continuous word embedding*

Let $\mathcal{V} = \{w_1, w_2, \ldots, w_V\}$ denote a finite vocabulary of $V = |\mathcal{V}|$ distinct discrete symbolic tokens. A **continuous word embedding** is an injective mapping:

$$\phi : \mathcal{V} \longrightarrow \mathbb{R}^d, \quad \text{with } d \ll V, \tag{1.3}$$

that maps each discrete symbol $w_i \in \mathcal{V}$ to a low-dimensional, dense continuous vector $\mathbf{v}_i = \phi(w_i) \in \mathbb{R}^d$. The target vector space $(\mathbb{R}^d, \|\cdot\|, \langle\cdot,\cdot\rangle)$ is a continuous metric space equipped with an inner product, where geometric distance $D(\mathbf{v}_i, \mathbf{v}_j)$ and angular orientation $\cos(\mathbf{v}_i, \mathbf{v}_j)$ preserve semantic and syntactic relationships.

In typical natural language and software engineering applications, vocabulary sizes range from $V \approx 30{,}000$ to upwards of $V = 1{,}000{,}000$ unique tokens, while the latent embedding dimension is chosen to be compact: $d \in [100, 1024]$ (e.g., $d = 300$ in classical Word2Vec and FastText; $d = 768$ in standard Transformer baselines). By mapping discrete words into $\mathbb{R}^d$, language tokens become continuous variables that participate directly in matrix algebra, linear transformations, and gradient backpropagation.

To understand why dense continuous representations are indispensable, we must first examine the standard categorical baseline: local one-hot encoding. While intuitive and trivial to construct, one-hot vectors suffer from severe geometric and computational pathologies that fundamentally incapacitate neural learning.

1.2 The baseline approach: local one-hot encodings and why they fail

The most straightforward, historical baseline for converting discrete categorical symbols into numeric vectors is **one-hot encoding**. Under this local representational scheme, every word $w_i \in \mathcal{V}$ is mapped to a standard basis coordinate vector $\mathbf{e}_i$ in a V-dimensional Euclidean space $\mathbb{R}^V$:

$$\mathbf{e}_i = [\underbrace{0,\dots,0}_{i-1 \text{ zeros}},1,\underbrace{0,\dots,0}_{V-i \text{ zeros}}]^\top \in \{0,1\}^V, \quad \text{where } (\mathbf{e}_i)_k = \delta_{ik} = \begin{cases} 1 & \text{if } k = i, \\ 0 & \text{if } k \neq i. \end{cases} \tag{1.4}$$

For example, consider a canonical toy vocabulary of $V = 5$ tokens:

$$\mathcal{V} = \{\texttt{"dog"}, \texttt{"cat"}, \texttt{"fish"}, \texttt{"pen"}, \texttt{"pencil"}\}. \tag{1.5}$$

Under one-hot encoding, each word is assigned an isolated Cartesian coordinate axis:

$$\mathbf{e}_{\text{dog}} = \begin{bmatrix}1\\0\\0\\0\\0\end{bmatrix}, \quad \mathbf{e}_{\text{cat}} = \begin{bmatrix}0\\1\\0\\0\\0\end{bmatrix}, \quad \mathbf{e}_{\text{fish}} = \begin{bmatrix}0\\0\\1\\0\\0\end{bmatrix}, \quad \mathbf{e}_{\text{pen}} = \begin{bmatrix}0\\0\\0\\1\\0\end{bmatrix}, \quad \mathbf{e}_{\text{pencil}} = \begin{bmatrix}0\\0\\0\\0\\1\end{bmatrix}. \tag{1.6}$$

1.2.1 Algebraic equivalence: one-hot matrix multiplication as embedding lookup

In the foundational guide *Build a Large Language Model from Scratch* [**?**], it is proven that feeding a one-hot vector into a dense linear layer is algebraically identical to retrieving a specific row or column from a weight lookup table.

Theorem 1.1: *Embedding lookup equivalence*

Let $\mathcal{V}$ be a vocabulary of size V, and let token $w_i \in \mathcal{V}$ be encoded as the standard basis vector $\mathbf{e}_i \in \{0,1\}^V$.

Column-vector orientation (standard basis representation)

Let $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d\times V}$ be the embedding matrix whose k-th column corresponds to the d-dimensional embedding vector $\mathbf{w}_k = (\mathbf{W}_{\text{in}})_{:,k} \in \mathbb{R}^d$:

$$\mathbf{W}_{\text{in}} = \begin{bmatrix} | & | & & | \\ \mathbf{w}_1 & \mathbf{w}_2 & \cdots & \mathbf{w}_V \\ | & | & & | \end{bmatrix}. \tag{1.7}$$

Multiplying $\mathbf{W}_{\text{in}}$ by the one-hot column vector $\mathbf{e}_i$ extracts the i-th column of $\mathbf{W}_{\text{in}}$:

$$\mathbf{h} = \mathbf{W}_{\text{in}}\mathbf{e}_i = \sum_{k=1}^{V} (\mathbf{e}_i)_k (\mathbf{W}_{\text{in}})_{:,k} = 1 \cdot (\mathbf{W}_{\text{in}})_{:,i} + \sum_{k\neq i} 0 \cdot (\mathbf{W}_{\text{in}})_{:,k} = \mathbf{w}_i \in \mathbb{R}^d. \tag{1.8}$$

Row-vector orientation (PyTorch convention)

Let $\mathbf{W} \in \mathbb{R}^{V\times d}$ be an embedding table whose k-th row contains the embedding vector $\mathbf{w}_k^\top = \mathbf{W}_{k,:} \in \mathbb{R}^{1\times d}$. Multiplying the one-hot row vector $\mathbf{e}_i^\top$ by $\mathbf{W}$ extracts the i-th row of $\mathbf{W}$:

$$\mathbf{e}_i^\top \mathbf{W} = \sum_{k=1}^{V} (\mathbf{e}_i)_k \mathbf{W}_{k,:} = 1 \cdot \mathbf{W}_{i,:} + \sum_{k\neq i} 0 \cdot \mathbf{W}_{k,:} = \mathbf{W}_{i,:} = \mathbf{w}_i^\top \in \mathbb{R}^{1\times d}. \tag{1.9}$$

Hence, matrix multiplication of an embedding weight matrix by a one-hot indicator vector is identical to an indexed memory table lookup:

$$\mathbf{W}_{\text{in}}\mathbf{e}_i \equiv \texttt{lookup}(\mathbf{W}_{\text{in}}, i). \tag{1.10}$$

1.2.2 Systems-level justification: why frameworks use dedicated embedding lookups

Although dense matrix multiplication $\mathbf{e}_i^\top \mathbf{W}$ and table indexing $\texttt{lookup}(\mathbf{W}, i)$ are algebraically equivalent, no modern deep learning framework (PyTorch, TensorFlow, JAX) ever implements word embeddings as a dense matrix multiplication. Deep learning frameworks provide specialized lookup modules, such as `torch.nn.Embedding(num_embeddings=V, embedding_dim=d)`, for three decisive systems engineering reasons:

1. **Elimination of $O(V \cdot d)$ Wasted FLOPs**: Evaluating the matrix multiplication $\mathbf{e}_i^\top \mathbf{W}$ requires executing $V \cdot d$ floating-point multiplications and $(V-1) \cdot d$ additions. In a standard language model with vocabulary $V = 50{,}257$ (GPT-2 tokenizer) and hidden dimension $d = 768$, computing the input projection for a single token via dense matrix multiplication requires:

   $$50{,}257 \times 768 \approx 3.86 \times 10^7 \text{ arithmetic operations per token.} \tag{1.11}$$

   Of these 38.6 million operations, exactly 99.998% evaluate $0 \times W_{k,j} = 0$. By contrast, `torch.nn.Embedding` accepts integer token IDs $i \in \{0, \ldots, V-1\}$ and performs direct $O(1)$ pointer arithmetic into physical GPU/CPU memory:

   $$\text{address}(\mathbf{w}_i) = \text{base_pointer} + i \times d \times \texttt{sizeof}(\text{float32}), \tag{1.12}$$

   retrieving the d-dimensional embedding slice in $O(1)$ time with exactly zero arithmetic FLOPs.

2. **Massive VRAM Savings (25,000× Memory Reduction)**: Consider a standard training batch with batch size $B = 64$ and sequence context length $T = 1{,}024$. If input sequences were materialized as dense one-hot tensors, the input tensor $\mathbf{X}_{\text{onehot}}$ would have shape $(B, T, V) = (64, 1{,}024, 50{,}257)$. Storing this tensor in single-precision floating point (`float32`, 4 bytes per element) would require:

   $$\text{VRAM}_{\text{one-hot}} = 64 \times 1{,}024 \times 50{,}257 \times 4 \text{ bytes} \approx 13{,}175{,}095{,}296 \text{ bytes} \approx 13.18\,\text{GB of VRAM,} \tag{1.13}$$

   simply to hold the input batch before computing a single activation. Conversely, storing sequences as integer token IDs produces an `int64` tensor of shape $(B, T) = (64, 1{,}024)$, requiring:

   $$\text{VRAM}_{\text{indices}} = 64 \times 1{,}024 \times 8 \text{ bytes} = 524{,}288 \text{ bytes} \approx 524\,\text{KB of VRAM.} \tag{1.14}$$

   This constitutes a memory bandwidth reduction factor of $\frac{13.18\,\text{GB}}{524\,\text{KB}} \approx \mathbf{25{,}128}\times$, entirely preventing Out-Of-Memory (OOM) GPU crashes.

3. **Sparse Gradient Backpropagation**: Under dense matrix multiplication $\mathbf{y} = \mathbf{e}_i^\top \mathbf{W}$, the backpropagation adjoint with respect to the weight matrix is the outer product:

   $$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{e}_i \left(\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \right) \in \mathbb{R}^{V \times d}. \tag{1.15}$$

   This gradient matrix is massive ($V \times d$) and strictly sparse: only row i contains non-zero values, while the remaining $V-1$ rows are populated entirely by zeros. Synchronizing such dense gradient tensors across multi-GPU distributed clusters saturates PCIe and NVLink bandwidth. `torch.nn.Embedding` executes sparse indexed gradient accumulation (`index_add_`), routing the upstream vector $\frac{\partial \mathcal{L}}{\partial \mathbf{y}} \in \mathbb{R}^d$ directly into the i-th row of $\mathbf{W}$.

1.2.3 The 5 structural failure modes of one-hot representations

While the lookup table implementation rescues computational throughput, local one-hot representations fail catastrophically as semantic representations. They suffer from five fatal pathologies:

Extreme sparsity and curse of dimensionality

As vocabulary size scales to web scale ($V \geq 50{,}000$ to 10^6), one-hot vectors become astronomical sparse vectors where 99.998% of entries are zero. The dimensionality of the vector space equals the number of distinct words. Under this curse of dimensionality, the volume of the enclosing space grows exponentially, leaving data points isolated at the extreme vertices of independent axes.

Metric collapse and semantic orthogonality

Because one-hot vectors form the standard Cartesian basis of $\mathbb{R}^V$, every pair of distinct words $w_i \neq w_j$ is mutually perpendicular and equidistant:

$$\cos(\mathbf{e}_i, \mathbf{e}_j) = \frac{\mathbf{e}_i^\top \mathbf{e}_j}{\|\mathbf{e}_i\|_2 \|\mathbf{e}_j\|_2} = \frac{0}{1 \cdot 1} = 0, \quad \forall i \neq j, \tag{1.16}$$

$$d_E(\mathbf{e}_i, \mathbf{e}_j) = \|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{(1)^2 + (-1)^2 + \sum_{k \neq i,j} 0^2} = \sqrt{2} \approx 1.414, \quad \forall i \neq j. \tag{1.17}$$

Under one-hot encoding, every word is orthogonal and equidistant to every other word:

$$d_E(\texttt{"pen"}, \texttt{"pencil"}) = d_E(\texttt{"pen"}, \texttt{"crocodile"}) = \sqrt{2}. \tag{1.18}$$

The underlying metric space collapses into a degenerate topology where open balls $B_r(\mathbf{e}_i) = \{\mathbf{x} \in \mathcal{V} : d_E(\mathbf{e}_i, \mathbf{x}) < r\}$ contain only $\{\mathbf{e}_i\}$ for $r \leq \sqrt{2}$, and immediately engulf the entire universe of vocabulary tokens for $r > \sqrt{2}$. There is zero geometric capacity to reflect semantic proximity, synonymy, or grammatical category.

Zero gradient parameter sharing during backpropagation

Because one-hot vectors activate strictly disjoint coordinates, training a neural network on an occurrence of token w_i produces gradients exclusively for the parameters associated with index i:

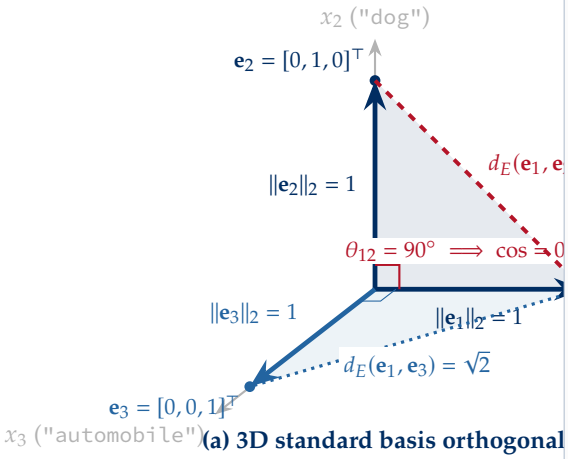
$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{k,:}} = \mathbf{0}, \quad \forall k \neq i. \tag{1.19}$$

If a model updates its weights upon observing the sentence *"The feline climbed the tree"*, the parameters associated with `"feline"` receive gradient updates, but the parameters for `"cat"`, `"kitten"`, or `"leopard"` remain completely unchanged. The model cannot transfer statistical strength across semantically synonymous or related words; it must encounter every word hundreds of times independently in every grammatical context.

Simplex confinement and absence of latent manifolds

One-hot vectors are confined strictly to the V extreme vertices of the standard $(V-1)$-dimensional probability simplex:

$$\Delta^{V-1} = \left\{ \mathbf{x} \in \mathbb{R}^V : \sum_{i=1}^{V} x_i = 1, \; x_i \geq 0 \right\}. \tag{1.20}$$



(a) 3D standard basis orthogonal

Pairwise metric equality matrices:

• **Cosine similarity matrix** (Gramian $\mathbf{E}^\top\mathbf{E} = \mathbf{I}_V$):

$$\mathbf{S}_{\cos} = \begin{pmatrix} & \text{cat} & \text{dog} & \text{car} \\ \text{cat} & 1 & \mathbf{0} & \mathbf{0} \\ \text{dog} & \mathbf{0} & 1 & \mathbf{0} \\ \text{car} & \mathbf{0} & \mathbf{0} & 1 \end{pmatrix}$$

• **Euclidean distance matrix** ($\mathbf{D}_{ij} = \sqrt{2}(1 - \delta_{ij})$):

$$\mathbf{D}_{L_2} = \begin{pmatrix} & \text{cat} & \text{dog} & \text{car} \\ \text{cat} & 0 & \mathbf{\sqrt{2}} & \mathbf{\sqrt{2}} \\ \text{dog} & \mathbf{\sqrt{2}} & 0 & \mathbf{\sqrt{2}} \\ \text{car} & \mathbf{\sqrt{2}} & \mathbf{\sqrt{2}} & 0 \end{pmatrix}$$

The geometric pathology:
dist("cat", "dog") = dist("cat", "car") = $\sqrt{2}$
cos("cat", "dog") = cos("cat", "car") = 0

(b) Semantic metric collapse

All tokens are equidistant and mutually perpendicular, completely destroying semantic metric topology.

Figure 1.1: Visual proof of pairwise orthogonality and equidistance in one-hot vector representations. (a) In V-dimensional Euclidean space, each word is an orthogonal standard basis vector of unit norm ($\|\mathbf{e}_i\|_2 = 1$). By the Pythagorean theorem, every distinct pair forms a right isosceles triangle with origin $\mathbf{0}$, yielding an identical Euclidean distance of $\sqrt{1^2 + 1^2} = \sqrt{2}$ and an inner product of 0 ($\theta = 90°$). (b) The resulting Gramian matrix is the identity $\mathbf{I}_V$, treating related concepts like `"cat"` and `"dog"` with the exact same geometric distance as totally unrelated concepts like `"cat"` and `"automobile"`.

The interior and edges of the simplex correspond to fractional probabilistic mixtures, but distinct words themselves inhabit isolated, non-overlapping boundary corners. Words cannot reside on smooth, lower-dimensional semantic submanifolds, nor can they support linear translation arithmetic ($\mathbf{w}_{\text{king}} - \mathbf{w}_{\text{man}} + \mathbf{w}_{\text{woman}} \approx \mathbf{w}_{\text{queen}}$).

Software engineering identifier polymorphism

In software engineering source code, developers routinely express identical semantic intents through differing lexical notations, identifier casings, and API conventions:

- **Casing variants**: `getUserById` vs. `get_user_by_id` vs. `GetUserById`.
- **API synonyms**: `push()` vs. `append()` vs. `add()` vs. `enqueue()`.
- **Syntactic equivalents**: Type references such as `ArrayList` and `LinkedList` vs. unrelated punctuation symbols like `";"`.

Under one-hot encoding, each casing variation and API synonym is assigned a totally independent, mutually orthogonal basis vector. To a one-hot model:

$$\cos(\mathbf{e}_{\texttt{get_user_by_id}}, \mathbf{e}_{\texttt{getUserById}}) = 0, \quad d_E(\mathbf{e}_{\texttt{ArrayList}}, \mathbf{e}_{\texttt{LinkedList}}) = d_E(\mathbf{e}_{\texttt{ArrayList}}, \mathbf{e}_{\texttt{";"}}) = \sqrt{2}. \tag{1.21}$$

This lexical fragmentation completely prevents code-intelligence models from transferring refactoring patterns, bug-fix templates, or type semantics across codebases.

Because local one-hot encodings confine words to isolated, orthogonal coordinate axes where semantic similarity is geometrically undefinable, we require a fundamentally different representational paradigm. This paradigm shift rests on distributing linguistic meaning across continuous latent dimensions—guided by the foundational Distributional Hypothesis.

1.3 Distributed representations and the distributional hypothesis

To overcome the orthogonality and isolation of local representations, modern deep learning adopts **distributed representations**, a concept formalized in connectionist neural networks in 1986 [**?**].

Definition 1.2: *Local vs. distributed representations*

- **Local Representation**: Each concept or symbol is represented by a single dedicated computational unit (e.g., one coordinate axis in $\mathbb{R}^V$). If that unit is inactive, the concept is entirely absent. There is zero overlap between distinct symbols.
- **Distributed Representation**: Each concept is represented by a continuous pattern of activation across all d dimensions of a shared vector space $\mathbb{R}^d$ ($d \ll V$). Conversely, each individual dimension participates in representing many distinct concepts.

In a distributed representation, meaning is not stored in an isolated slot; rather, it emerges from the continuous coordinates along latent dimensions. For example, latent dimensions might implicitly reflect attributes such as animacy, size, sentiment, syntactic part of speech, or software typing constructs.

1.3.1 The distributional hypothesis: context as meaning

How can an optimization algorithm discover meaningful continuous coordinates $\mathbf{v}_w \in \mathbb{R}^d$ for every word purely from raw text, without human supervision? The answer lies in the **Distributional Hypothesis**, formulated in structural linguistics in 1954 [**?**] and popularized in 1957 [**?**]:

> *"You shall know a word by the company it keeps."*
> — Distributional Hypothesis [**?**]

The distributional hypothesis posits that words that appear in similar linguistic contexts tend to share similar semantic and pragmatic meanings. Consider two sentences:

Sentence 1: "I used a **pencil** to write the essay."
Sentence 2: "You used my **pen** to write a letter."

Even if a machine has never encountered a dictionary definition of `"pencil"` or `"pen"`, it observes that both tokens co-occur with identical context words: `"used"`, `"write"`, `"essay"`, `"letter"`. Conversely, an unrelated word such as `"crocodile"` or `"refrigerator"` almost never occurs in the context of `"used a [...] to write"`.

Therefore, statistical co-occurrence within local contextual windows serves as an empirical, computable proxy for semantic similarity. If a learning algorithm is tasked with predicting surrounding context words from a target word (or vice versa), the learned vector coordinates must position words with similar co-occurrence profiles close together in vector space.

While the Distributional Hypothesis asserts that context defines meaning, it does not specify an optimization algorithm. To translate linguistic co-occurrence into optimized vector coordinates, we must frame a self-supervised prediction task and establish an end-to-end differentiable training loop.

1.4 The embedding training loop and semantic geometry evolution

To convert the distributional hypothesis into concrete numerical representations, we formulate a **self-supervised prediction cloze task** from unannotated text corpora.

1.4.1 The 7-step self-supervised optimization cycle

Given a raw, unannotated corpus of T running words $C = (w_1, w_2, \ldots, w_T)$, we shift a sliding context window of half-width C across the text. At each step t, the model solves a fill-in-the-blank prediction task: estimating the probability distribution over the vocabulary for a missing token given its surrounding context:

$$\mathbb{P}(w_t = w \mid w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C}). \tag{1.22}$$

This self-supervised prediction task is executed through an iterative **7-step optimization cycle**:

1. **Initialize Parameter Matrices**: Allocate two weight matrices with small random values: the input embedding matrix $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$ and the output embedding matrix $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$.

2. **Context Lookup and Aggregation**: For the active context window, retrieve the input embedding vectors $\mathbf{v}_c = (\mathbf{W}_{\text{in}})_{:,c}$ and aggregate them into a single hidden context vector $\mathbf{h} \in \mathbb{R}^d$ (e.g., via vector summation or averaging).

3. **Output Candidate Projection**: Compute the dot product between the hidden context state $\mathbf{h}$ and every candidate output word vector $\mathbf{u}_w = (\mathbf{W}_{\text{out}})_{:,w}$ in the vocabulary:

$$z_w = \mathbf{u}_w^\top \mathbf{h}, \quad \text{yielding logit vector } \mathbf{z} = \mathbf{W}_{\text{out}}^\top \mathbf{h} \in \mathbb{R}^V. \tag{1.23}$$

4. **Probability Normalization**: Pass the similarity logits through the softmax activation function to produce a valid probability distribution $\hat{\mathbf{y}} \in \Delta^{V-1}$:

$$\hat{y}_w = \mathbb{P}(w \mid \text{context}) = \frac{\exp(z_w)}{\sum_{k=1}^{V} \exp(z_k)} = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})}. \tag{1.24}$$

5. **Loss Evaluation**: Evaluate the prediction error against the true ground-truth target token $w^* = w_t$ using Categorical Cross-Entropy:

$$\mathcal{L}_{\text{CE}} = -\log \hat{y}_{w^*} = -\log \left(\frac{\exp(\mathbf{u}_{w^*}^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})} \right). \tag{1.25}$$

6. **Backpropagation Error Adjoints**: Apply reverse-mode automatic differentiation to compute error adjoints. The gradient with respect to logit vector $\mathbf{z}$ is the prediction residual $\boldsymbol{\delta} = \hat{\mathbf{y}} - \mathbf{y} \in \mathbb{R}^V$, where $\mathbf{y} = \mathbf{e}_{w^*}$. Propagating this residual backward yields parameter gradients:

$$\nabla_{\mathbf{W}_{\text{out}}} \mathcal{L} = \mathbf{h}\, \boldsymbol{\delta}^\top \in \mathbb{R}^{d \times V}, \quad \nabla_{\mathbf{h}} \mathcal{L} = \mathbf{W}_{\text{out}} \boldsymbol{\delta} \in \mathbb{R}^d, \quad \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L} = (\nabla_{\mathbf{h}} \mathcal{L})\, \mathbf{e}_{\text{context}}^\top \in \mathbb{R}^{d \times V}. \tag{1.26}$$

7. **Stochastic Gradient Descent Parameter Update**: Adjust the coordinates in both embedding matrices by stepping along the negative gradient direction:

$$\mathbf{W}_{\text{out}} \leftarrow \mathbf{W}_{\text{out}} - \eta \nabla_{\mathbf{W}_{\text{out}}} \mathcal{L}, \qquad \mathbf{W}_{\text{in}} \leftarrow \mathbf{W}_{\text{in}} - \eta \nabla_{\mathbf{W}_{\text{in}}} \mathcal{L}. \tag{1.27}$$

Figure 1.2 illustrates this complete learning mechanism, juxtaposing the forward/backward computational dataflow in Panel (a) with the resulting geometric evolution of the embedding space across training epochs in Panel (b).

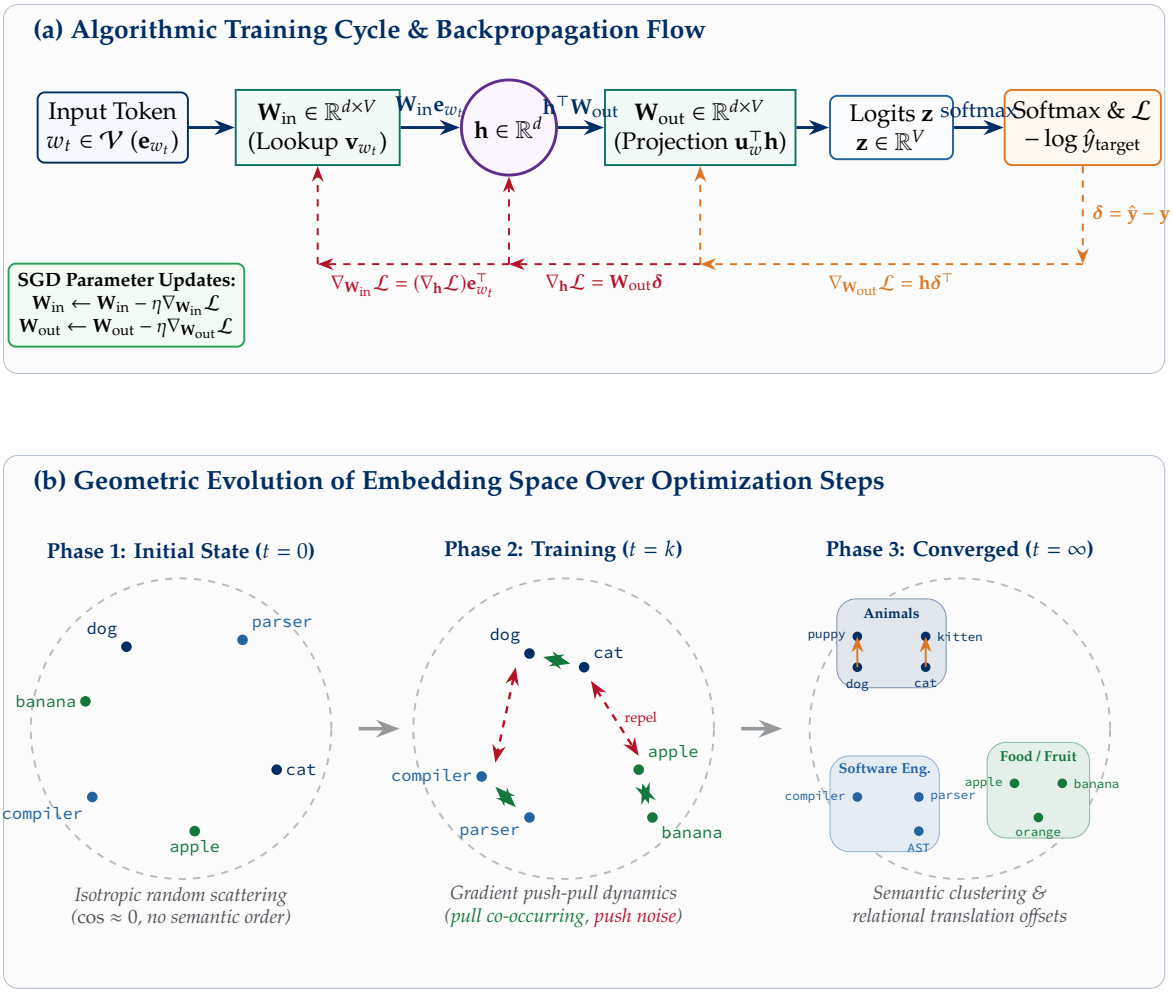


Figure 1.2: The embedding training loop and semantic space evolution. (a) Computational dataflow of the forward prediction pass and backward adjoint parameter updates. (b) Three-phase geometric evolution of embedding space from isotropic random scattering to clustered semantic topology with parallel relational offsets.

1.4.2 Geometric phase transition of the embedding space

As shown in Panel (b) of Figure 1.2, the continuous embedding space undergoes a profound topological self-organization over the course of training:

- **Phase 1: Initial State ($t = 0$).** Vectors are initialized using small Gaussian noise, $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I}_d)$ with $\sigma \approx 0.01$ or $\sigma \approx 1/\sqrt{d}$. In high dimensions ($d \geq 100$), randomly sampled vectors are almost exactly orthogonal ($\cos \approx 0$). Tokens from disparate semantic domains—`"dog"`, `"compiler"`, `"apple"`—are scattered isotropically across the unit sphere with zero structural order.
- **Phase 2: Training Dynamics ($t = k$).** Whenever the model processes an observed sentence (e.g., *"The compiler parsed the syntax tree"*), backpropagation produces an attractive gradient force pulling the vectors of co-occurring words closer together. Concurrently, negative gradients or normalizer updates exert repulsive forces, pushing non-co-occurring words apart.
- **Phase 3: Converged Semantic Topology ($t = \infty$).** Upon convergence across millions of sentences, words coalesce into dense semantic neighborhoods (e.g., domestic animals, compiler construction terms, fruits). Remarkably, linear vector differences between related concepts become parallel across distinct pairs, giving rise to relational vector arithmetic (e.g., $\mathbf{v}_{\text{dog}} - \mathbf{v}_{\text{puppy}} \approx \mathbf{v}_{\text{cat}} - \mathbf{v}_{\text{kitten}}$).

1.4.3 Initialization dynamics and the hockey-stick loss curve

In educational analyses of neural language models and lookup tables [**?**], it is demonstrated that the scale of initial weights in $\mathbf{W}_{\text{in}}$ and $\mathbf{W}_{\text{out}}$ dictates optimization stability. At initialization, before the network has learned any linguistic patterns, the model should assign an approximately uniform probability to every word in the vocabulary:

$$\hat{y}_w \approx \frac{1}{V}, \quad \forall w \in \mathcal{V}. \tag{1.28}$$

Consequently, the theoretical expected initial cross-entropy loss is strictly:

$$\mathcal{L}_{\text{init}} = -\log\left(\frac{1}{V}\right) = \log V. \tag{1.29}$$

For a vocabulary of $V = 50{,}000$, the initial loss must be $\mathcal{L}_{\text{init}} = \ln(50{,}000) \approx 10.82$. If $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$ are initialized with excessively large variance, logits $z_w = \mathbf{u}_w^\top \mathbf{h}$ take extreme values; the softmax distribution becomes peaky and confident on arbitrary words, causing the initial loss to spike drastically ($\mathcal{L} \gg 20$). The optimizer must then spend thousands of iterations flattening these unearned confidences—producing a pathological "hockey-stick" loss curve. Proper variance scaling ($\mathcal{N}(0, 0.01^2)$ or $\mathcal{N}(0, 1/d)$) ensures stable, well-conditioned gradient flow from step zero.

With the general self-supervised training cycle established, we must concretize the network architecture. The first foundational architecture is the Continuous Bag-of-Words (CBOW) model, which aggregates surrounding context words to predict an omitted target center word.

1.5 Word2Vec continuous bag-of-words (CBOW)

In 2013, **Word2Vec** was introduced by Mikolov et al. [**?** **?**] as a suite of log-linear models designed to scale distributed representation learning to web-scale corpora containing billions of tokens.

The first of these architectures is the **Continuous Bag-of-Words (CBOW)** model. In CBOW, the objective is to predict the omitted center target token w_t given the symmetric surrounding context window:

$$C_t = \{w_{t-C}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+C}\} \setminus \{w_t\}. \tag{1.30}$$

1.5.1 Matrix formulation and tensor plumbing

The network consists of two weight matrices:

- $\mathbf{W}_{\text{in}} \in \mathbb{R}^{d \times V}$: The input embedding matrix, where each column $(\mathbf{W}_{\text{in}})_{:,k} = \mathbf{v}_k \in \mathbb{R}^d$ represents the vector of token k when acting as context.
- $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times V}$: The output embedding matrix, where each column $(\mathbf{W}_{\text{out}})_{:,k} = \mathbf{u}_k \in \mathbb{R}^d$ represents the vector of token k when acting as the target.

Let $\mathbf{e} \in \{0,1\}^V$ denote the multi-hot context vector indicating the presence of words in C_t:

$$e_k = \sum_{j \in \{-C,\ldots,C\} \setminus \{0\}} \mathbf{1}_{\{w_{t+j} = w_k\}}. \tag{1.31}$$

The hidden state $\mathbf{h} \in \mathbb{R}^d$ is obtained by projecting the context vector through $\mathbf{W}_{\text{in}}$ and averaging over the $2C$ context positions:

$$\mathbf{h} = \frac{1}{2C}\mathbf{W}_{\text{in}}\mathbf{e} = \frac{1}{2C} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \mathbf{v}_{w_{t+j}} \in \mathbb{R}^d. \tag{1.32}$$

Next, the model scores candidate target words by projecting $\mathbf{h}$ against the columns of $\mathbf{W}_{\text{out}}$:

$$\hat{\mathbf{p}} = \mathbf{h}^\top \mathbf{W}_{\text{out}} \in \mathbb{R}^{1 \times V}, \quad \text{where } \hat{p}_k = \mathbf{u}_k^\top \mathbf{h}. \tag{1.33}$$

As emphasized in geometric visual analyses [**?**], the inner product $\mathbf{u}_k^\top \mathbf{h} = \|\mathbf{u}_k\|_2 \|\mathbf{h}\|_2 \cos\theta$ acts as a directional alignment sensor: it evaluates how closely the target word's representation points in the same direction as the accumulated context vector.

The raw similarity logits $\hat{\mathbf{p}}$ are normalized via the softmax function to form the conditional probability distribution:

$$\mathbb{P}(w_t = w \mid C_t) = \frac{\exp(\mathbf{u}_w^\top \mathbf{h})}{\sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h})}. \tag{1.34}$$

The model is trained by minimizing the negative log-likelihood of the ground-truth target word $w^* = w_t$:

$$\mathcal{L}_{\text{CBOW}} = -\log \mathbb{P}(w_t = w^* \mid C_t) = -\mathbf{u}_{w^*}^\top \mathbf{h} + \log \sum_{k=1}^{V} \exp(\mathbf{u}_k^\top \mathbf{h}). \tag{1.35}$$

Figure 1.3 illustrates this computational flow.

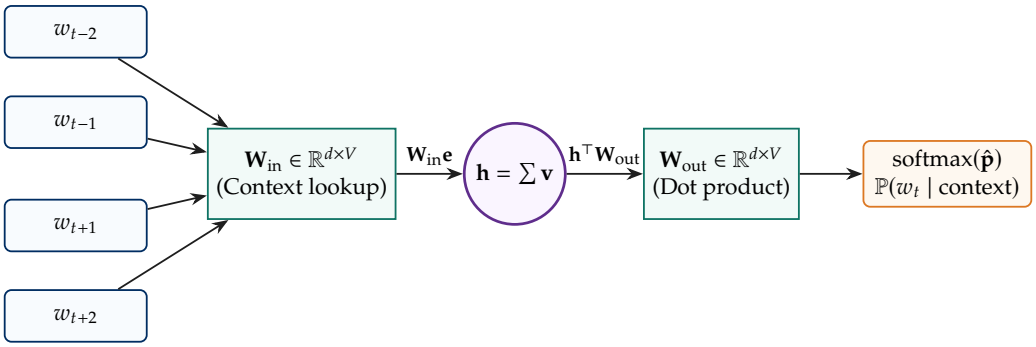


Figure 1.3: Continuous Bag-of-Words (CBOW) architecture. Context words $w_{t-c}, \ldots, w_{t+c}$ are looked up in $\mathbf{W}_{\text{in}}$ and summed to form context state $\mathbf{h} \in \mathbb{R}^d$. Inner products with output matrix $\mathbf{W}_{\text{out}}$ yield similarity logits normalized via softmax to predict the center target token w_t.

1.5.2 Comparison with earlier neural language models: the log-linear breakthrough

A neural language model learning distributed embeddings had been introduced a decade prior in 2003 [**?**]. However, that architecture had two major differences:

1. **Strict Causal Prediction**: In [**?**], language was modeled autoregressively, predicting w_t strictly from past tokens $w_{t-n+1}, \ldots, w_{t-1}$. In Word2Vec, the window was opened symmetrically to both left and right context tokens $(w_{t-C}, \ldots, w_{t+C})$, doubling the statistical signal.

2. **Elimination of the Non-Linear Hidden Layer**: In [**?**], concatenated context embeddings were passed through a non-linear dense hidden layer $\tanh(\mathbf{W}_h\mathbf{x} + \mathbf{b})$. The matrix multiplication with $\mathbf{W}_h$ dominated computation time ($O(C \cdot d \cdot d_h)$). In Word2Vec, the non-linear activation was stripped away entirely, using a direct linear projection (vector averaging). This made Word2Vec a log-linear model, reducing compute costs by orders of magnitude and enabling training on billions of words in a few hours.

1.5.3 Strengths and representational bottlenecks of CBOW

Strengths of CBOW

- **Rapid Training Convergence**: Context averaging smooths the gradient signal, allowing CBOW to converge in fewer epochs over large corpora.

- **Robust Representations for Frequent Words**: For high-frequency tokens with abundant contextual observations, the smoothed averaging stabilizes coordinate estimates against noisy outliers.

Representational bottlenecks of CBOW

- **Loss of Sequential Word Order (Syntactic Blindness)**: Vector summation in Equation (1.32) is strictly commutative: $\mathbf{v}_a + \mathbf{v}_b = \mathbf{v}_b + \mathbf{v}_a$. The model is incapable of distinguishing *"the dog bit the man"* from *"the man bit the dog"*.

- **Rare Word Dilution**: Because context words are averaged into a single vector $\mathbf{h}$, if a rare technical token appears alongside multiple high-frequency stopwords (e.g., *"the **polymorphism** in the"*), the rare token's embedding is diluted by the dominant stopwords: $\frac{1}{2C}\mathbf{v}_{\text{rare}} \approx \mathbf{0}$. The gradient update to the rare word is severely attenuated.

Although CBOW trains rapidly by averaging context representations, this very averaging creates a representational bottleneck: it washes out rare vocabulary tokens and discards syntactic word order. To learn high-fidelity representations for infrequent words, inverting the predictive formulation yields the Continuous Skip-Gram architecture.

1.6 Word2Vec continuous skip-gram and the softmax bottleneck

To preserve the distinct gradient signal of rare words, the **Continuous Skip-Gram** architecture was introduced in [**?**]. Instead of aggregating context words to predict a single center token, Skip-Gram **inverts the predictive direction**: given a single center target word w_t, the model independently predicts each surrounding context word within window radius C.

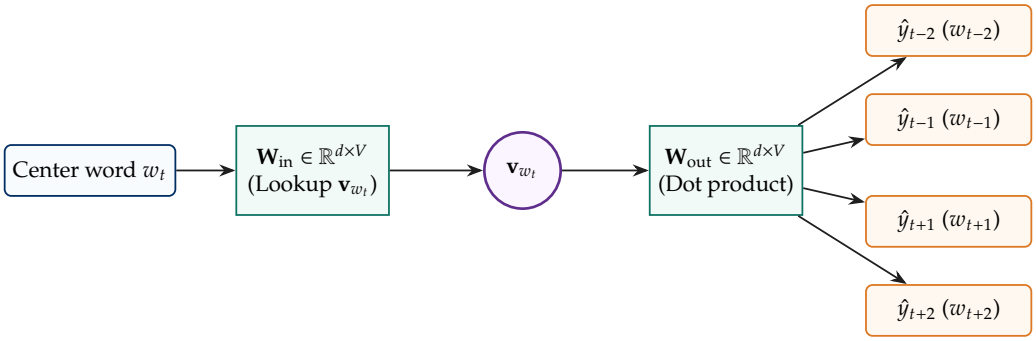


Figure 1.4: Continuous Skip-Gram architecture. A single center target word w_t is projected via $\mathbf{W}_{\text{in}}$ into vector $\mathbf{v}_{w_t} \in \mathbb{R}^d$, which predicts the probability distribution of context words across window radius c through output matrix $\mathbf{W}_{\text{out}}$.

Definition 1.3: *Skip-gram objective function*

Given a training sequence of T words $w_1, w_2, \ldots, w_T$ and a context window of half-width C, the Skip-gram model maximizes the average log-likelihood:

$$\mathcal{L}_{\text{SG}} = \frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-C \le j \le C \\ j \neq 0}} \log \mathbb{P}(w_{t+j} \mid w_t). \tag{1.36}$$

Under standard multinomial softmax, the conditional probability of context word w_O given center word w_I is:

$$\mathbb{P}(w_O \mid w_I) = \frac{\exp\left(\mathbf{u}_{w_O}^\top \mathbf{v}_{w_I}\right)}{\sum_{w=1}^{V} \exp\left(\mathbf{u}_w^\top \mathbf{v}_{w_I}\right)}, \tag{1.37}$$

where $\mathbf{v}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{in}}$ (input embedding) and $\mathbf{u}_w \in \mathbb{R}^d$ denotes the column of $\mathbf{W}_{\text{out}}$ (output context embedding).

1.6.1 Why Skip-gram captures rare words

In Skip-Gram, context words are never averaged together. Instead, every context-target co-occurrence (w_I, w_O) generates an independent, dedicated pairwise training instance. When an infrequent, domain-specific term (such as `"refactoring"` or `"polymorphism"`) appears in text, its vector $\mathbf{v}_{w_I}$ directly predicts each context word:

$$\nabla_{\mathbf{v}_{w_I}} \mathcal{L} = \sum_{w_O \in C} \left(\mathbf{u}_{w_O} - \sum_{k=1}^{V} \mathbb{P}(w_k \mid w_I) \mathbf{u}_k \right). \tag{1.38}$$

The gradient is never diluted by surrounding words. Consequently, Skip-Gram produces significantly higher-quality embeddings for rare words, specialized technical jargon, and

nuanced semantic distinctions than CBOW.

1.6.2 The shared computational wall: the full softmax bottleneck

Despite its empirical superiority, the standard formulation of Skip-Gram crashes into an insurmountable computational wall: **the full Softmax partition function bottleneck**.

Examining the denominator of Equation (1.37):

$$Z(w_I) = \sum_{w=1}^{V} \exp\left(\mathbf{u}_w^\top \mathbf{v}_{w_I}\right). \tag{1.39}$$

Computing this normalizing denominator requires evaluating an inner product $\mathbf{u}_w^\top \mathbf{v}_{w_I}$ and an exponential $\exp(\cdot)$ for **every single token w in the entire vocabulary $\mathcal{V}$**.

- **Computational Complexity per Token**: Computing $Z(w_I)$ takes $O(V \cdot d)$ floating-point operations.
- **Total Corpus Cost**: For a window of radius $C = 5$, predicting $2C = 10$ context words per token over a corpus of $T = 10^9$ words with vocabulary $V = 10^5$ and dimension $d = 300$ requires:

$$10^9 \text{ tokens} \times 10 \text{ predictions} \times 10^5 \text{ words} \times 300 \text{ dims} = 3 \times 10^{17} \text{ FLOPs}. \tag{1.40}$$

- **Gradient Synchronization Bottleneck**: In backpropagation, the derivative $\frac{\partial \mathcal{L}}{\partial \mathbf{u}_w}$ touches every column $w \in \{1, \ldots, V\}$ of $\mathbf{W}_{\text{out}}$ on every single training step, forcing full-matrix memory writes.

This $O(V \cdot d)$ complexity renders standard multinomial softmax completely intractable for web-scale corpora.

While Continuous Skip-Gram excels at capturing nuanced semantic relationships and rare tokens, its standard formulation crashes into a severe computational barrier: normalizing over the full vocabulary requires an $O(V \cdot d)$ denominator calculation for every single token prediction. To scale to realistic vocabularies, we must circumvent this partition function bottleneck.

1.7 Scaling softmax: hierarchical softmax and negative sampling

To bypass the $O(V \cdot d)$ partition function barrier, two distinct scaling methods were formulated in [**?**]: **Hierarchical Softmax** (a structured tree-based approximation) and **Skip-Gram with Negative Sampling (SGNS)** (a sample-based binary discrimination formulation).

1.7.1 Hierarchical softmax ($O(\log_2 V)$)

First developed in 2005 [**?**] and adapted in [**?**], **Hierarchical Softmax** decomposes the flat multiclass probability distribution into a hierarchical sequence of binary routing decisions over a binary tree.

- **Binary Tree Topology**: The V vocabulary words form the leaves of a binary tree (constructed as a binary Huffman tree where frequent words have shorter code lengths).
- **Path Specification**: For any word $w \in \mathcal{V}$, let $L(w)$ denote the depth of the leaf from the root. There exists a unique path of nodes from the root node $n(w, 1) =$ root to the leaf node $n(w, L(w)) = w$.
- **Internal Node Parameters**: Each internal node $j \in \{1, \ldots, L(w)-1\}$ is assigned a continuous parameter vector $\mathbf{v}'_{n(w,j)} \in \mathbb{R}^d$.
- **Binary Branching Probability**: At each internal node, the probability of branching to the left child is parameterized by the logistic sigmoid function:

$$\mathbb{P}(\text{branch left} \mid \mathbf{v}_{w_I}) = \sigma\left(\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right) = \frac{1}{1 + \exp\left(-\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right)}. \tag{1.41}$$

  The probability of branching to the right child is:

$$\mathbb{P}(\text{branch right} \mid \mathbf{v}_{w_I}) = 1 - \sigma\left(\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right) = \sigma\left(-\mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right). \tag{1.42}$$

- **Path Probability Product**: The conditional probability of reaching leaf word w is the product of binary decision probabilities along its unique path:

$$\mathbb{P}(w \mid w_I) = \prod_{j=1}^{L(w)-1} \sigma\left(\text{sign}(w, j) \cdot \mathbf{v}'^{\top}_{n(w,j)} \mathbf{v}_{w_I}\right), \tag{1.43}$$

  where $\text{sign}(w, j) = +1$ if the path branches left at node j, and -1 if it branches right.

Because probabilities sum to 1 at each binary branching step ($\sigma(z) + \sigma(-z) = 1$), telescoping products guarantee that the sum of probabilities over all leaves equals 1 identically:

$$\sum_{w \in \mathcal{V}} \mathbb{P}(w \mid w_I) \equiv 1. \tag{1.44}$$

No global partition function $Z(w_I)$ ever needs to be computed!

Evaluating $\mathbb{P}(w \mid w_I)$ and its gradients requires traversing only the $L(w) - 1$ nodes along the path. In a balanced or Huffman binary tree, the average path length is $O(\log_2 V)$. For a vocabulary of $V = 100{,}000$:

$$\log_2(100{,}000) \approx 16.6 \implies \frac{100{,}000}{17} \approx \mathbf{5{,}880}\times \text{ acceleration}. \tag{1.45}$$

1.7.2 Skip-gram negative sampling (SGNS)

While Hierarchical Softmax scales to $O(\log_2 V)$, an even more efficient and empirically superior alternative was formulated in [**?**]: **Negative Sampling (SGNS)**, derived as a simplified approximation to Noise Contrastive Estimation (NCE).

Instead of treating context prediction as a massive multinomial classification task over V mutually exclusive categories, SGNS reformulates the problem as $1 + K$ **independent binary logistic classifications**:

1. **Positive Sample ($y = 1$):** The true center-context pair (w_c, w_o) observed in the text corpus. The model maximizes the probability that this pair is a true co-occurrence.
2. **Negative Samples ($y = 0$):** K noise words $\{w_{n_1}, w_{n_2}, \ldots, w_{n_K}\}$ drawn at random from a noise distribution $P_n(w)$. The model maximizes the probability that these randomly sampled pairs are synthetic noise.

Definition 1.4: *Skip-gram negative sampling objective*

For an observed center-context token pair (w_c, w_o) and K negative samples $\{w_{n_1}, \ldots, w_{n_K}\} \sim P_n(w)$, the SGNS loss is:

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma\left(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}\right) - \sum_{k=1}^{K} \log \sigma\left(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}\right), \tag{1.46}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ denotes the standard logistic sigmoid function.

1.7.3 Step-by-step display math analytical gradient derivations

To understand how SGNS shapes the geometry of the embedding space, we derive the exact analytical gradients of $\mathcal{L}_{\text{SGNS}}$ with respect to all parameter vectors.

Recall the fundamental identities for the logistic sigmoid derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)), \tag{1.47}$$

$$\frac{d}{dz} \log \sigma(z) = \frac{\sigma'(z)}{\sigma(z)} = 1 - \sigma(z), \tag{1.48}$$

$$\frac{d}{dz} \log \sigma(-z) = \frac{-\sigma'(-z)}{\sigma(-z)} = \frac{-\sigma(-z)(1 - \sigma(-z))}{\sigma(-z)} = -(1 - \sigma(-z)) = -\sigma(z). \tag{1.49}$$

Gradient with respect to positive target vector

The positive target vector $\mathbf{u}_{w_o}$ appears exclusively in the first term of Equation (1.46):

$$\begin{aligned} \frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} &= -\frac{\partial}{\partial \mathbf{u}_{w_o}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \\ &= -\left[1 - \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})\right] \frac{\partial(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_o}} \\ &= \left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1\right] \mathbf{v}_{w_c}. \end{aligned} \tag{1.50}$$

Defining the positive prediction error as $e_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1 = \hat{y}_{\text{pos}} - 1$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_o}} = (\hat{y}_{\text{pos}} - 1)\mathbf{v}_{w_c}. \tag{1.51}$$

Gradient with respect to negative noise vector

Each negative sample vector $\mathbf{u}_{w_{n_k}}$ appears in exactly one term of the summation in Equation (1.46):

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} &= -\frac{\partial}{\partial \mathbf{u}_{w_{n_k}}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= -\left[-\sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\right] \frac{\partial(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})}{\partial \mathbf{u}_{w_{n_k}}} \\ &= \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\, \mathbf{v}_{w_c}. \end{aligned} \tag{1.52}$$

Defining the negative prediction error as $e_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) - 0 = \hat{y}_{\text{neg},k}$, this simplifies to:

$$\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{u}_{w_{n_k}}} = \hat{y}_{\text{neg},k}\, \mathbf{v}_{w_c}. \tag{1.53}$$

Gradient with respect to center word vector

The center word vector $\mathbf{v}_{w_c}$ appears in both the positive term and all K negative terms. Applying the chain rule to the complete loss:

$$\begin{aligned}\frac{\partial \mathcal{L}_{\text{SGNS}}}{\partial \mathbf{v}_{w_c}} &= -\frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - \sum_{k=1}^{K} \frac{\partial}{\partial \mathbf{v}_{w_c}} \log \sigma(-\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \\ &= \underbrace{\left[\sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) - 1\right] \mathbf{u}_{w_o}}_{\text{Attractive Component (Pull Context)}} + \underbrace{\sum_{k=1}^{K} \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c})\, \mathbf{u}_{w_{n_k}}}_{\text{Repulsive Component (Push Noise)}}. \end{aligned} \tag{1.54}$$

1.7.4 Physical vector mechanics interpretation

Under gradient descent, parameter updates subtract the gradient: $\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} - \eta \nabla_{\mathbf{v}_{w_c}} \mathcal{L}$:

$$\mathbf{v}_{w_c} \leftarrow \mathbf{v}_{w_c} + \eta \left(1 - \hat{y}_{\text{pos}}\right) \mathbf{u}_{w_o} - \eta \sum_{k=1}^{K} \hat{y}_{\text{neg},k}\, \mathbf{u}_{w_{n_k}}. \tag{1.55}$$

This update formula admits a clean, intuitive **physical vector mechanics** interpretation:

- **The Attractive Pull Force**: Since $\hat{y}_{\text{pos}} = \sigma(\mathbf{u}_{w_o}^\top \mathbf{v}_{w_c}) \in (0, 1)$, the coefficient $(1 - \hat{y}_{\text{pos}})$ is strictly positive. The update adds a vector component in the direction $+\mathbf{u}_{w_o}$, **pulling the center word vector toward the true context vector**. If the model is already confident ($\hat{y}_{\text{pos}} \approx 1$), the force vanishes; if the model is surprised ($\hat{y}_{\text{pos}} \approx 0$), the force reaches maximum strength $+1 \cdot \mathbf{u}_{w_o}$.
- **The Repulsive Push Force**: For every negative sample k, $\hat{y}_{\text{neg},k} = \sigma(\mathbf{u}_{w_{n_k}}^\top \mathbf{v}_{w_c}) \in (0, 1)$ is strictly positive. The update subtracts a vector component along each noise vector $-\mathbf{u}_{w_{n_k}}$, **pushing the center word vector away from noise words**. If the noise word is far away ($\hat{y}_{\text{neg},k} \approx 0$), no force is exerted; if the noise word is erroneously close ($\hat{y}_{\text{neg},k} \approx 1$), a strong repulsive kick $-\mathbf{u}_{w_{n_k}}$ repels the points.

Crucially, the computational complexity of evaluating Equation (1.55) is $O(K \cdot d)$, which is **entirely independent of vocabulary size** V. In practice, choosing $K \in [5, 20]$ for small corpora and $K \in [2, 5]$ for web-scale corpora provides sufficient negative contrast, reducing training time from days to hours.

1.7.5 Theoretical justification of the $3/4$ unigram noise exponent

How should negative samples be drawn? In [**?**], the noise distribution $P_n(w)$ is defined as a smoothed unigram distribution raised to the power of $\alpha = 3/4 = 0.75$:

$$P_n(w) = \frac{f(w)^{3/4}}{\sum_{w' \in \mathcal{V}} f(w')^{3/4}}, \tag{1.56}$$

where $f(w)$ denotes the raw unigram occurrence frequency of token w in the corpus.

Theoretical justification of the 3/4 unigram noise exponent

1. **Failure of Raw Frequency ($\alpha = 1.0$)**: Natural language word frequencies follow Zipf's law ($f(r) \propto 1/r$). A tiny fraction of stopwords (`"the"`, `"of"`, `"and"`, `"to"`) account for over 50% of all token occurrences in a typical corpus. If $\alpha = 1.0$, more than half of all negative samples drawn are uninformative stopwords. The model wastes its entire discriminative budget separating words from `"the"`, while rare content words (`"polymorphism"`, `"compiler"`) have near-zero probability of ever being selected as negative samples, receiving virtually zero negative gradient updates.

2. **Failure of Uniform Sampling ($\alpha = 0.0$)**: Under a uniform distribution $P_n(w) = 1/V$, every vocabulary word has equal probability. Because the vast majority of words in any natural vocabulary reside in the long tail of rare words, sampling uniformly selects almost exclusively rare words as negatives. The network rarely practices distinguishing moderately frequent content words from one another.

3. **The Sub-linear Compromise ($\alpha = 0.75$)**: The exponent 3/4 acts as a concave, sub-linear dampener. It compresses the gap between ultra-frequent stopwords and rare content words.

Consider a frequent stopword w_{freq} with corpus frequency $f(w_{\text{freq}}) = 0.01$ (e.g., 10^6 occurrences in 10^8 words) and a rare technical term w_{rare} with corpus frequency $f(w_{\text{rare}}) = 10^{-6}$ (e.g., 100 occurrences in 10^8 words):

- Under raw unigram frequency ($\alpha = 1.0$), the sampling ratio is:

$$\frac{f(w_{\text{freq}})}{f(w_{\text{rare}})} = \frac{10^{-2}}{10^{-6}} = 10{,}000. \tag{1.57}$$

  The stopword is sampled 10,000 times more frequently than the rare word.

- Under the smoothed distribution ($\alpha = 0.75$), the ratio becomes:

$$\frac{f(w_{\text{freq}})^{0.75}}{f(w_{\text{rare}})^{0.75}} = \left(\frac{10^{-2}}{10^{-6}}\right)^{0.75} = (10{,}000)^{0.75} = (10^4)^{3/4} = 10^3 = 1{,}000. \tag{1.58}$$

By applying the 3/4 power, the relative probability of sampling the rare word is amplified by a factor of:

$$\frac{10{,}000}{1{,}000} = \mathbf{10}\times. \tag{1.59}$$

This 10× boost provides rare words with sufficient negative gradient pushes to carve out accurate decision boundaries, while still reflecting broad corpus statistics.

1.7.6 Global count matrix factorization: GloVe

Around the same time, **GloVe (Global Vectors for Word Representation)** was introduced in [**?**]. Whereas Word2Vec is a local window-based predictive model that processes streaming word pairs, GloVe demonstrated that distributed embeddings could be trained by directly factorizing a **global co-occurrence matrix** $\mathbf{X} \in \mathbb{R}^{V \times V}$, where X_{ij} counts the number of times word j appears in the context window of word i.

GloVe's core insight is that the semantic distinction between two words i and j relative to a probe word k is encoded not by raw probabilities $P_{ik} = X_{ik}/X_i$, but by the **ratio of co-occurrence probabilities**:

$$\frac{P_{ik}}{P_{jk}} = \frac{P(k \mid i)}{P(k \mid j)}. \tag{1.60}$$

Table 1.1 reproduces the seminal demonstration from [**?**] comparing i = `"ice"` and j = `"steam"` across probe words k.

Table 1.1: Co-occurrence probability ratios illustrating semantic discrimination in GloVe [**?**].

Probability & Ratio	k = `"solid"`	k = `"gas"`	k = `"water"`	k = `"fashion"`
$P(k \mid$ `"ice"`$)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \mid$ `"steam"`$)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \mid$ `"ice"`$)/P(k \mid$ `"steam"`$)$	**8.9**	**0.085**	1.36	0.96

When k = `"solid"` is strongly related to `"ice"` but not `"steam"`, the ratio is large ($8.9 \gg 1$). When k = `"gas"` is related to `"steam"` but not `"ice"`, the ratio is tiny ($0.085 \ll 1$). When k = `"water"` is equally related to both (or k = `"fashion"` is related to neither), the ratio hovers near 1.0.

GloVe enforces this relationship by fitting a log-bilinear model where dot products approximate log co-occurrence counts, minimized via weighted least squares:

$$J = \sum_{i=1}^{V} \sum_{j=1}^{V} f(X_{ij}) \left(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2, \tag{1.61}$$

where $f(X_{ij}) = \min(1, (X_{ij}/x_{\max})^\alpha)$ is a sub-linear weighting function preventing frequent stopwords from dominating the loss. Both Word2Vec and GloVe construct semantically structured vector spaces, with SGNS operating on streaming text and GloVe operating on aggregated count statistics.

Skip-Gram Negative Sampling and GloVe resolve the computational bottleneck, making large-scale distributed representation learning feasible. However, Word2Vec operates under an implicit architectural assumption: that the word is the minimal, indivisible atomic unit of language. In real-world deployment, this assumption creates severe structural vulnerabilities.

1.8 Limitations of Word2Vec and the out-of-vocabulary (OOV) barrier

Despite their revolutionary impact, classical Word2Vec and GloVe architectures suffer from fundamental linguistic and software engineering limitations.

1.8.1 The atomic word token assumption and out-of-vocabulary (OOV) failure

Word2Vec models assume a closed, fixed vocabulary $\mathcal{V} = \{w_1, \ldots, w_V\}$ established during training. Each word is treated as an indivisible, atomic entity: its character composition is completely ignored. This creates the **Out-of-Vocabulary (OOV) catastrophe**:

- **Unseen Words at Test Time**: In open-domain text and evolving codebases, users constantly produce novel tokens: technical neologisms, composite identifiers (e.g., `AsyncHttpClientFactory`), misspelled words (e.g., `"recusive"`), and inflected stems.
- **Mapping to `<UNK>`**: When a trained Word2Vec model encounters an OOV word at inference, it has no entry in $\mathbf{W}_{\text{in}}$ or $\mathbf{W}_{\text{out}}$. Deep learning pipelines are forced to map all OOV tokens to a generic unknown token placeholder:

$$w_{\text{oov}} \longmapsto \texttt{<UNK>} \implies \mathbf{v}_{w_{\text{oov}}} = \mathbf{v}_{\texttt{<UNK>}}. \tag{1.62}$$

  This discards all semantic identity. To Word2Vec, a typo like `"algorithmm"` is as completely devoid of meaning as an alien string.

1.8.2 Morphological blindness

Word2Vec is entirely blind to internal word morphology. In natural languages with rich inflectional systems (e.g., Italian, German, Turkish) as well as programming languages, words share sub-morphemic stems, prefixes, and suffixes:

Natural language: `"develop"`, `"developer"`, `"developing"`, `"undeveloped"`,
Software identifiers: `"parseFile"`, `"parseTokens"`, `"parseTree"`.

Under Word2Vec, each of these variants is assigned a totally disjoint, independent column in $\mathbf{W}_{\text{in}}$. The model possesses zero architectural mechanism to share parameters across words containing the common root `"develop"` or `"parse"`. If `"undeveloped"` occurs only twice in the corpus, its vector remains poorly trained, unable to inherit semantic properties from the frequent token `"develop"`.

1.8.3 Polysemy collapse: static context invariance

Word2Vec assigns exactly **one static vector $\mathbf{v}_w \in \mathbb{R}^d$ to each word string** w, regardless of the surrounding runtime context in which it appears. This results in **polysemy collapse**.

Definition 1.5: *Polysemy collapse*

Let word w possess M distinct semantic senses $\mathcal{S}_1, \ldots, \mathcal{S}_M$ with respective corpus mixture proportions $\pi_1, \ldots, \pi_M$ ($\sum \pi_m = 1$). Because Word2Vec minimizes prediction error across all occurrences using a single vector $\mathbf{v}_w$, the optimal vector converges to the convex combination of distinct sense centroids:

$$\mathbf{v}_w^* \approx \sum_{m=1}^{M} \pi_m \mathbf{v}_{\mathcal{S}_m}. \tag{1.63}$$

For example, consider the polysemous word `"bat"`:

Sense 1 (Zoological): "A **bat** is a nocturnal flying mammal."
Sense 2 (Sports Equipment): "The baseball player swung the wooden **bat**."

In Word2Vec, $\mathbf{v}_{\text{bat}}$ is pulled toward biological animals on some iterations, and toward sports equipment on others. The resulting point converges to an intermediate centroid in $\mathbb{R}^d$ that represents neither animal nor sports gear accurately, degrading cosine similarity to both senses.

In software engineering, polysemy is ubiquitous:

- `"Python"`: A biological serpent vs. a high-level dynamic programming language.
- `"Spring"`: A season of the year vs. a popular Java enterprise application framework.
- `"rust"`: Iron oxide corrosion vs. a memory-safe systems programming language.

A static Word2Vec model maps `"Rust"` to a compromise vector that conflates oxidation chemistry with memory safety guarantees.

While contextual ambiguity requires sequence models that dynamically modulate representations based on full sentence context, the Out-of-Vocabulary barrier and morphological blindness can be solved at the token representation level. By abandoning the word as an atomic unit and descending into subword character n-grams, FastText establishes a morphologically aware embedding paradigm.

1.9 FastText: subword character n-gram embeddings

To conquer the OOV barrier and enable morphological parameter sharing, **FastText** was introduced by Bojanowski et al. [**?**].

1.9.1 Character n-gram decomposition and boundary markers

Rather than treating a word as an atomic token, FastText represents each word as a **bag of character n-grams**.

1. **Boundary Enclosure**: To distinguish prefixes and suffixes from character sequences occurring in the interior of a word, each word w is bracketed by special boundary symbols `"<"` and `">"`:
$$\tilde{w} = \texttt{"<"} \circ w \circ \texttt{">"}. \tag{1.64}$$
This ensures that the prefix sequence `"<re"` (as in `"<refactor>"`) is mathematically distinct from the internal substring `"re"` (as in `"<where>"`).

2. **Subword Extraction**: All character n-grams of length $n \in [n_{\min}, n_{\max}]$ (typically $n \in [3, 6]$) are extracted, along with the special sequence containing the entire word itself $\tilde{w}$.

Subword decomposition worked example

Consider the word `"where"` decomposed into character trigrams ($n = 3$):

$$\texttt{"<where>"} \implies \mathcal{G}_{\texttt{where}} = \Big\{\texttt{"<wh"}, \texttt{"whe"}, \texttt{"her"}, \texttt{"ere"}, \texttt{"re>"}, \texttt{"<where>"}\Big\}. \tag{1.65}$$

1.9.2 Vector sum composition

Each subword n-gram g is assigned a latent continuous embedding vector $\mathbf{z}_g \in \mathbb{R}^d$. The final representation of word w is defined as the **sum of its constituent subword vectors**:

$$\mathbf{v}_w = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g \in \mathbb{R}^d. \tag{1.66}$$

Under the Skip-Gram Negative Sampling objective, the similarity score between center word w and context word c becomes:

$$s(w, c) = \mathbf{v}_w^\top \mathbf{u}_c = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g^\top \mathbf{u}_c. \tag{1.67}$$

During backpropagation, the prediction error with respect to $\mathbf{v}_w$ distributes gradients to every constituent subword vector $\mathbf{z}_g$.

1.9.3 Resolving OOV tokens dynamically and morphological sharing in code

This simple compositional formulation yields two decisive capabilities:

1. **Zero-Shot Synthesis of Out-of-Vocabulary Tokens**: When an unseen word w_{oov} appears at test time (e.g., a compound identifier like `"refactorable"` or a code typo like `"recusive"`), even if the full token was never seen during training, its constituent character n-grams (`"<re"`, `"ref"`, `"fact"`, `"able>"`) were updated across other vocabulary words. FastText

synthesizes an accurate embedding vector on the fly by summing the vectors of its known subwords:

$$\mathbf{v}_{w_{\text{oov}}} = \sum_{g \in \mathcal{G}_{w_{\text{oov}}}} \mathbf{z}_g. \tag{1.68}$$

The OOV barrier is completely eliminated!

2. **Morphological Parameter Sharing**: In natural language, prefixes like `"un-"`, suffixes like `"-tion"`, and inflections like `"-ing"` accumulate shared vector representations across hundreds of distinct words. In software engineering, identifier substrings automatically share semantics:

$$\texttt{"parseXmlDocument"} \quad \text{and} \quad \texttt{"parseJsonDocument"} \tag{1.69}$$

share the character subwords for `"parse"` and `"Document"`. The model immediately recognizes that both identifiers describe parsing operations on structured documents.

Memory bounding via the Fowler-Noll-Vo hash trick

Because the universe of all character n-grams ($n \in [3, 6]$) across large vocabularies exceeds tens of millions of unique strings, allocating an unbounded matrix would cause memory blowout. FastText bounds the embedding table to $K = 2 \times 10^6$ rows using the Fowler-Noll-Vo (FNV-1a) hashing function:

$$\text{index}(g) = \text{FNV-1a}(g) \pmod{K}. \tag{1.70}$$

Colliding n-grams share bucket rows; due to the high dimensionality ($d = 300$) and independent sum aggregation, hash collisions act as benign regularizing noise that does not disrupt semantic retrieval.

1.9.4 Connection to modern subword tokenizers: BPE and WordPiece

As highlighted in pedagogical breakdowns of language models [**?**], the subword philosophy pioneered by FastText directly inspired the tokenization mechanisms powering modern Large Language Models:

- **Byte Pair Encoding (BPE)** [**? ?**]: Iteratively merges the most frequent pairs of adjacent characters or bytes to form a deterministic subword vocabulary (used in GPT-2, GPT-4, and LLaMA).
- **SentencePiece** [**?**]: Treats input text as a raw byte stream without whitespace pre-segmentation, training unigram language model segmentations.

FastText operates continuous character n-gram summations at the embedding layer, whereas BPE operates discrete token mergers during preprocessing. Both architectures fundamentally exist to dismantle the brittle, isolated word-level assumption and establish robust subword parameter sharing.

By decomposing words into character n-grams, FastText conquers the Out-of-Vocabulary barrier and enables morphological parameter sharing. With our embedding models fully trained, we can now step back and examine the remarkable mathematical geometry that spontaneously emerges within these dense vector spaces.

1.10 Emergent vector space geometry: clustering, metrics and relational analogies

When Word2Vec and FastText models are trained over large text corpora, the resulting continuous vector spaces exhibit profound emergent mathematical properties: high-dimensional semantic clustering, metric equivalence, and linear relational translations.

1.10.1 Semantic clustering and dimensionality reduction

In high-dimensional embedding space ($d = 300$), words with similar functional roles organize into distinct, tightly bound clusters.

Projecting 300-dimensional FastText vectors into two dimensions via Principal Component Analysis (PCA) reveals distinct semantic clusters:

- **Household Items**: `"table"`, `"chair"`, `"desk"`, `"sofa"`.
- **Mammals**: `"dog"`, `"cat"`, `"tiger"`, `"elephant"`.
- **Birds**: `"eagle"`, `"sparrow"`, `"parrot"`, `"pigeon"`.

The leading principal components preserve categorical variance: domestic entities, animal taxonomy, and biological sub-branches form linearly separable clusters in the compressed 2D projection as well as in the original 300-dimensional manifold.

1.10.2 Linear relational translations (vector analogy arithmetic)

The most celebrated emergent property of continuous word embeddings is that semantic and syntactic relationships manifest as **linear vector translation offsets**:

$$\mathbf{v}_{w_2} - \mathbf{v}_{w_1} \approx \vec{d}_{\text{relation}}. \tag{1.71}$$

Country-to-capital linear offsets

For instance, subtracting the vector for `"Berlin"` from `"Germany"` produces a directional displacement vector $\vec{d}_{\text{capital_of}}$:

$$\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}} \approx \vec{d}_{\text{capital_of}}. \tag{1.72}$$

Applying this learned offset vector to another country predicts its capital city via simple vector addition:

$$\mathbf{v}_{\text{Spain}} + (\mathbf{v}_{\text{Germany}} - \mathbf{v}_{\text{Berlin}}) \approx \mathbf{v}_{\text{Madrid}}. \tag{1.73}$$

The gender-royalty parallelogram

The canonical demonstration established in [**?**] is the gender-royalty parallelogram:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}. \tag{1.74}$$

Figure 1.5 illustrates this parallelogram in two dimensions.

Software engineering relational analogies

In code representation learning (e.g., CodeBERT [**?**], OpenAI Codex embeddings [**?**]), relational vector arithmetic reflects runtime architectures and language toolchains:

$$\mathbf{v}_{\text{Java}} - \mathbf{v}_{\text{JVM}} + \mathbf{v}_{\text{Python}} \approx \mathbf{v}_{\text{CPython}}, \tag{1.75}$$

$$\mathbf{v}_{\text{C++}} - \mathbf{v}_{\text{g++}} + \mathbf{v}_{\text{Rust}} \approx \mathbf{v}_{\text{rustc}}. \tag{1.76}$$

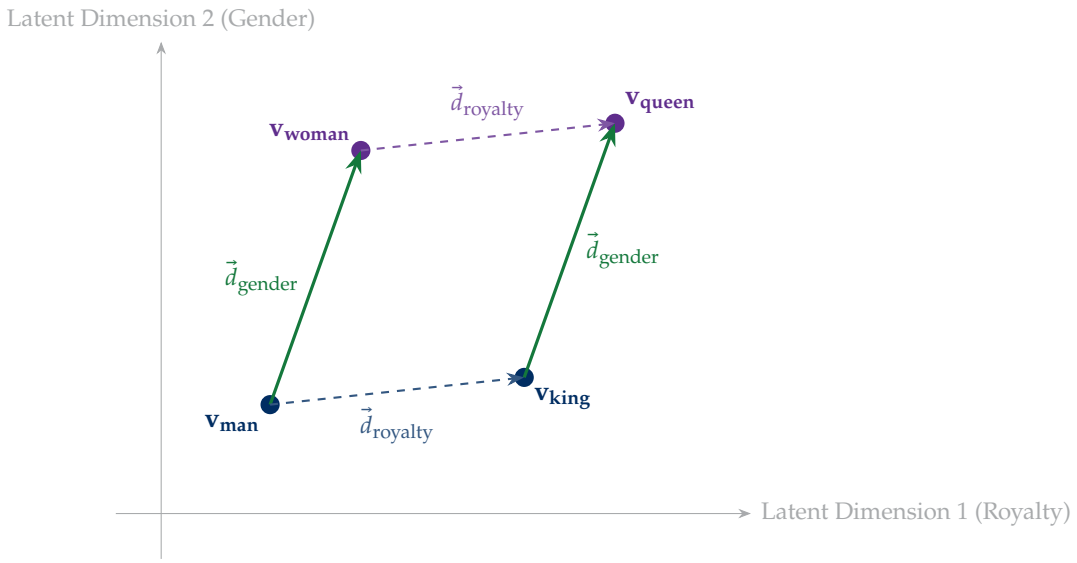


Figure 1.5: Geometric parallelogram in continuous embedding space demonstrating relational vector translations: $\mathbf{v}_{\text{King}} - \mathbf{v}_{\text{Man}} \approx \mathbf{v}_{\text{Queen}} - \mathbf{v}_{\text{Woman}}$. The directional displacement $\vec{d}_{\text{gender}}$ isolates the feminine gender transformation, while $\vec{d}_{\text{royalty}}$ isolates the monarchical status offset.

1.10.3 Metric geometry of embedding spaces: cosine vs. Euclidean distance

When querying nearest neighbors or evaluating semantic similarity between two continuous vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, three metrics are commonly considered:

1. **Euclidean (L_2) Distance**:

$$D_{L_2}(\mathbf{u}, \mathbf{v}) = \|\mathbf{u} - \mathbf{v}\|_2 = \sqrt{\sum_{i=1}^{d} (u_i - v_i)^2}. \tag{1.77}$$

2. **Dot Product (Inner Product)**:

$$\langle \mathbf{u}, \mathbf{v} \rangle = \mathbf{u}^\top \mathbf{v} = \sum_{i=1}^{d} u_i v_i = \|\mathbf{u}\|_2 \|\mathbf{v}\|_2 \cos\theta. \tag{1.78}$$

3. **Cosine Distance**:

$$D_{\cos}(\mathbf{u}, \mathbf{v}) = 1 - \cos(\mathbf{u}, \mathbf{v}) = 1 - \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \in [0, 2]. \tag{1.79}$$

Theorem 1.2: *Monotonic equivalence under L_2 normalization*

Let $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ be normalized to unit Euclidean length ($\|\mathbf{u}\|_2 = \|\mathbf{v}\|_2 = 1$). Then squared Euclidean distance is strictly proportional to Cosine distance:

$$\|\mathbf{u} - \mathbf{v}\|_2^2 = \|\mathbf{u}\|_2^2 + \|\mathbf{v}\|_2^2 - 2\mathbf{u}^\top \mathbf{v} = 1 + 1 - 2\cos(\mathbf{u}, \mathbf{v}) = 2(1 - \cos(\mathbf{u}, \mathbf{v})) = 2D_{\cos}(\mathbf{u}, \mathbf{v}). \tag{1.80}$$

Consequently, nearest neighbor rankings under Euclidean distance, Cosine distance, and Dot Product are mathematically identical on the unit hypersphere $\mathbb{S}^{d-1}$.

Why cosine similarity dominates metric evaluations

As demonstrated in educational analyses [**?**], when word embeddings are trained without explicit length regularization, the Euclidean norm $\|\mathbf{v}_w\|_2$ strongly correlates with unigram token frequency. Highly frequent words undergo millions of gradient pushes, systematically expanding their vector magnitude, whereas rare terms remain close to the origin. Cosine distance normalizes out vector magnitude, projecting all embeddings onto $\mathbb{S}^{d-1}$. It evaluates **pure angular orientation** (conceptual topic) rather than **vector length** (frequency artifact).

1.10.4 Dimensionality trade-offs and the Johnson-Lindenstrauss lemma

Why do word embeddings operate effectively in dimensions $d \in [100, 300]$, and modern LLMs in $d \in [768, 4096]$?

- **Under-parameterization ($d < 50$)**: Severe geometric crowding occurs. The space lacks enough orthogonal degrees of freedom to separate independent semantic concepts, forcing unrelated topics into spurious proximity.
- **Over-parameterization ($d \gg 10{,}000$)**: The space suffers from the curse of dimensionality. The volume of the unit ball concentrates entirely in a thin outer shell, distance distributions become uniform, and memory bandwidth demands explode.

The mathematical guarantee underwriting this stability is the **Johnson-Lindenstrauss Lemma**:

Theorem 1.3: *Johnson-Lindenstrauss Lemma*

Given $0 < \epsilon < 1$, any set of N points in arbitrary Euclidean space can be embedded into a Euclidean space of dimension:

$$d = O\left(\frac{\log N}{\epsilon^2}\right), \tag{1.81}$$

such that all pairwise Euclidean distances are preserved within a distortion factor of $1 \pm \epsilon$:

$$(1-\epsilon)\|\mathbf{u}-\mathbf{v}\|_2^2 \leq \|f(\mathbf{u}) - f(\mathbf{v})\|_2^2 \leq (1+\epsilon)\|\mathbf{u}-\mathbf{v}\|_2^2. \tag{1.82}$$

For a vocabulary of $N = 100{,}000$ tokens and distortion tolerance $\epsilon = 0.2$, $\log(10^5)/0.04 \approx 287$, precisely justifying why $d = 300$ serves as the golden standard for dense static word vectors.

1.11 Modern subword tokenization and input pipelines for foundation models

In Sections 1.1 through 1.10, we formulated the mathematical foundations of classical distributed word representations (Word2Vec, GloVe, FastText). While continuous embeddings solved the differentiability barrier and geometric similarity, classical architectures operated under a fragmented, static paradigm: input text was naively pre-split on whitespace into word-level units, vocabulary sizes ballooned into hundreds of thousands of entries, rare or misspelled tokens collapsed into the uninformative `<UNK>` placeholder, and input sequences were treated as unordered bags or static lookup tables.

Modern Large Language Models (LLMs) such as GPT-2, GPT-3, GPT-4, LLaMA, and Claude discard this isolated design in favor of an integrated, end-to-end **data preprocessing, tokenization, and input embedding pipeline**. As formalized in *Build a Large Language Model from Scratch* [**?**], preparing raw text for foundation model pretraining requires an indivisible four-stage architectural pipeline:

1. **Subword Tokenization**: Segmenting raw text into atomic subword units using Byte Pair Encoding (BPE), resolving punctuation and whitespace losslessly.
2. **Token ID Mapping**: Converting discrete subwords into numerical indices via an invertible vocabulary lookup table, enriched with special operational context markers.
3. **Continuous Embedding Projection**: Retrieving dense vector representations for each token index from a learned embedding table $\mathbf{W}_{\text{tok}} \in \mathbb{R}^{V \times d}$.
4. **Composite Positional Synthesis**: Injecting sequential order into permutation-invariant neural layers by additively combining continuous token embeddings with positional embeddings: $\mathbf{X}_{\text{in}} = \mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}}$.

Figure 1.6 illustrates this end-to-end data pipeline from raw textual strings to the final continuous tensor ingested by Transformer architectures.

1.11.1 The tokenization spectrum: character, word, and subword trade-offs

Converting raw natural language or software code into discrete units reveals a fundamental trade-off across three representational granularities: character-level, word-level, and subword tokenization.

Limitations of whitespace-based splitting

A naive baseline approach to text tokenization splits raw strings on whitespace characters (‘\s’). However, whitespace splitting fails catastrophically on punctuation:

```
"Hello, world!  This is a test."  ⟹ (whitespace split)  ['Hello,', 'world!', 'This', 'is',
                                                          'a', 'test.']
```

The comma is fused to `"Hello"`, and periods are fused to `"world"` and `"test"`. Consequently, the model assigns distinct vocabulary indices to `"world"`, `"world!"`, `"world,"`, and `"world?"`. This fragments training counts, bloats the vocabulary with redundant punctuation variants, and destroys syntactic generalization.

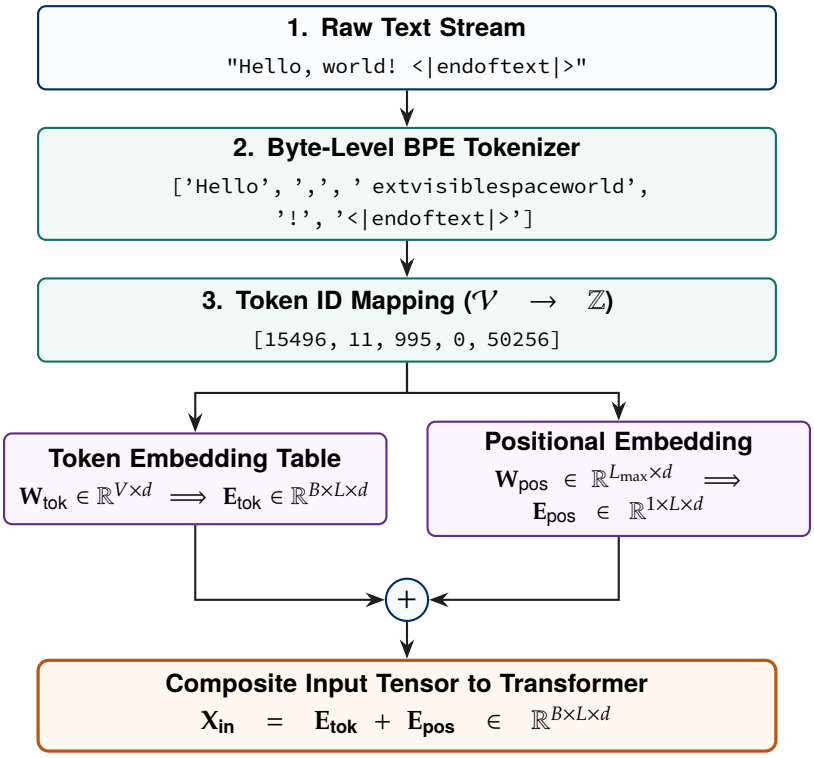


Figure 1.6: The modern foundation model tokenization and input preparation pipeline. Raw text is segmented into byte-level BPE subwords, mapped to unique integer token IDs, and converted into continuous latent vectors where token embeddings and positional embeddings combine additively.

Table 1.2: Comprehensive taxonomy and trade-off analysis of tokenization paradigms in natural language processing and modern foundation models.

Paradigm	Vocab (V)	Length (L)	OOV	Linguistic & Systems Limitations	Representative Models
Character-Level	100 – 256	Massive (4 – 6×)	Zero	High sequence length explodes computational complexity $O(L^2)$ in self-attention; lacks word-level semantic priors.	Char-RNN, Canine, ByT5
Word-Level	50k – 1M+	Minimal (1×)	Severe	Out-of-vocabulary words collapse to `<UNK>`; morphologically blind; massive embedding matrix $\mathbf{W} \in \mathbb{R}^{V \times d}$.	Word2Vec, GloVe, N-gram LMs
Subword-Level	32k – 100k	Moderate (1.2 – 1.4×)	Zero	Requires statistical offline pre-tokenization merge training; optimal Pareto frontier of length and vocabulary.	BPE (GPT-2/3/4, LLaMA), WordPiece (BERT)

Regex-based tokenization with punctuation isolation

To isolate punctuation while preserving them as distinct semantic tokens, a regular expression splitting rule can be formulated [**?**] that captures punctuation symbols, em-dashes, and whitespaces:

```python
import re

class SimpleTokenizerV1:
    def __init__(self, vocab):
        self.str_to_int = vocab
        self.int_to_str = {i: s for s, i in vocab.items()}

    def encode(self, text):
        # Split on whitespace, punctuation marks, and special dash patterns
        tokens = re.split(r'([,.:;?_!"()\']|--|\s)', text)
        tokens = [t.strip() for t in tokens if t.strip()]
        token_ids = [self.str_to_int[t] for t in tokens]
        return token_ids

    def decode(self, token_ids):
        text = " ".join([self.int_to_str[i] for i in token_ids])
        # Clean up spaces before punctuation marks
        text = re.sub(r'\s+([,.?!])', r'\1', text)
        return text

# Building the vocabulary from unique preprocessed tokens
text = "Hello, world. Is this-- a test?"
raw_tokens = re.split(r'([,.:;?_!"()\']|--|\s)', text)
tokens = [t.strip() for t in raw_tokens if t.strip()]
vocab = {token: idx for idx, token in enumerate(sorted(set(tokens)))}
tokenizer = SimpleTokenizerV1(vocab)
encoded = tokenizer.encode("Hello, world.")
print("Encoded IDs:", encoded)
print("Decoded text:", tokenizer.decode(encoded))
```

Listing 1.1: Basic regular expression text tokenization with punctuation isolation and vocabulary construction [**?**].

1.11.2 Special context tokens: document boundaries, padding, and masked batching

In practical machine learning and software engineering workflows, a vocabulary cannot consist solely of natural language words. The model requires **special context tokens** to encode structural and operational metadata:

1. **Unknown Word Token** (<|unk|>): In fixed-vocabulary systems, words unseen during training cannot be mapped to an integer index. Such tokens are mapped to <|unk|>. While this prevents code execution errors, it irreversibly destroys semantic information: a technical identifier or typo is flattened into a generic void.

2. **Document Boundary Delimiter** (<|endoftext|>): In large-scale autoregressive pretraining, billions of tokens spanning Wikipedia articles, books, and GitHub repositories are concatenated into continuous sequential streams. Without an explicit separator, the causal attention mechanism would assume that the first sentence of an article is the causal continuation of an unrelated previous document! The token <|endoftext|> signals that all preceding context is decoupled from subsequent tokens, allowing clean document demarcation.

3. **Padding Token** (<|pad|>): Mini-batch processing on GPU hardware requires rectangular tensors $\mathbf{X} \in \mathbb{Z}^{B \times L}$. When batching sequences of varying lengths ($L_1 \neq L_2$), shorter sequences are padded with <|pad|> tokens up to maximum length L. Crucially, the model's self-attention mechanism is configured with an **attention mask**:

$$M_{ij} = \begin{cases} 0 & \text{if token } j \text{ is a genuine text token,} \\ -\infty & \text{if token } j \text{ is a <|pad|> token.} \end{cases} \tag{1.83}$$

Adding M_{ij} to attention logits before Softmax drives pad attention weights to $\frac{e^{-\infty}}{\sum e^z} = 0$, completely preventing padded positions from corrupting token representations.

1.11.3 Byte Pair Encoding (BPE): algorithmic mechanics and merge trees

To conquer the Out-of-Vocabulary barrier without blowing up sequence length, modern LLMs deploy **Byte Pair Encoding (BPE)**. Originally invented for data compression in [**?**], BPE was adapted to natural language processing in [**?**].

BPE constructs a subword vocabulary through an iterative bottom-up statistical merge procedure partitioned into distinct training and inference phases:

1. **Base Alphabet Initialization**: Initialize base vocabulary $\mathcal{V}_0$ with all individual characters occurring across the training corpus C, augmenting each word with an explicit end-of-word boundary marker (e.g., '</w>'):

$$\mathcal{V}_0 = \{c \mid c \in w,\ w \in C\} \cup \{\texttt{</w>}\}. \tag{1.84}$$

Every word in C is initially split into a sequence of individual character symbols.

2. **Adjacent Pair Frequency Tally**: At iteration k, construct the co-occurrence frequency histogram over all adjacent symbol pairs (p_1, p_2) across all tokenized words in C:

$$(p_1^*, p_2^*) = \arg\max_{(p_1,p_2)} \sum_{w \in C} \text{count}(w) \cdot \mathbb{I}\{(p_1, p_2) \subseteq w\}. \tag{1.85}$$

3. **Greedy Merge and Priority Rule Logging**: Merge the globally most frequent pair (p_1^*, p_2^*) into an atomic new subword unit $p_{\text{new}} = p_1^* \circ p_2^*$. Crucially, record this merge operation in an ordered priority dictionary $\mathcal{M}$:

$$\mathcal{M}[k] = ((p_1^*, p_2^*) \mapsto p_{\text{new}}). \tag{1.86}$$

4. **In-Place Corpus Contraction**: Substitute every contiguous occurrence of (p_1^*, p_2^*) across corpus C with the merged symbol p_{new}, and expand the vocabulary:

$$\mathcal{V}_k = \mathcal{V}_{k-1} \cup \{p_{\text{new}}\}. \tag{1.87}$$

5. **Termination and Inference Application**: Repeat Steps 2–4 until reaching the target vocabulary budget $|\mathcal{V}_k| = V_{\text{target}}$. At inference time, any arbitrary input word is first split into its constituent base characters, and the learned merge rules in $\mathcal{M}$ are applied sequentially according to their training rank.

BPE training merge mechanics and vocabulary growth

To illustrate how BPE resolves out-of-vocabulary tokens without explosion, consider a small transparent corpus consisting of four words with their occurrence frequencies:

$$C = \{\texttt{"hug"} : 10,\ \texttt{"pug"} : 5,\ \texttt{"pun"} : 12,\ \texttt{"hugs"} : 5\}. \tag{1.88}$$

The total corpus count is $10 + 5 + 12 + 5 = 32$ word instances. The initial base vocabulary consists of 7 symbols: $\mathcal{V}_0 = \{\texttt{'h'}, \texttt{'u'}, \texttt{'g'}, \texttt{'p'}, \texttt{'n'}, \texttt{'s'}, \texttt{'</w>'}\}$. The segmented corpus begins as:

```
h u g </w> :  10    p u g </w> :  5    p u n </w> :  12    h u g s </w> :  5
```

- **Iteration 1**: We evaluate the frequency of all adjacent bigrams across the corpus:
  - ('u','g') : 10(hug) + 5(pug) + 5(hugs) = 20.
  - ('p','u') : 5(pug) + 12(pun) = 17.
  - ('h','u') : 10(hug) + 5(hugs) = 15.
  - ('g','</w>') : 10(hug) + 5(pug) = 15.
  - ('u','n') : 12(pun), ('n','</w>') : 12(pun).
  - ('g','s') : 5(hugs), ('s','</w>') : 5(hugs).

  The most frequent pair is ('u','g') with frequency 20. We log Merge Rule 1: ('u','g') → 'ug' and update $\mathcal{V}_1 = \mathcal{V}_0 \cup \{\texttt{'ug'}\}$. The corpus contracts to:

```
h ug </w> :  10    p ug </w> :  5    p u n </w> :  12    h ug s </w> :  5
```

- **Iteration 2**: We recompute adjacent pair frequencies with the merged symbol 'ug':
  - ('h','ug') : 10 + 5 = 15.
  - ('ug','</w>') : 10 + 5 = 15.
  - ('p','u') : 12(pun), ('u','n') : 12(pun), ('n','</w>') : 12(pun).
  - ('p','ug') : 5(pug), ('ug','s') : 5(hugs), ('s','</w>') : 5(hugs).

  The highest frequency is 15. Selecting ('h','ug'), we log Merge Rule 2: ('h','ug') → 'hug' and update $\mathcal{V}_2 = \mathcal{V}_1 \cup \{\texttt{'hug'}\}$. The corpus becomes:

```
hug </w> :  10    p ug </w> :  5    p u n </w> :  12    hug s </w> :  5
```

Inference segmentation walkthrough on unseen tokens

Now consider what happens when a model deployed in production encounters the word "pugs" (the plural of pug). Notice that "pugs" **never appeared in the training corpus as a complete word** (the training data only contained "pug" and "hugs").

1. Under a traditional word-level tokenizer, "pugs" is completely absent from vocabulary $\mathcal{V}$ and collapses into the information-destroying token <UNK>.
2. Under Byte Pair Encoding:
   - The word is initialized into base characters: ['p','u','g','s'].
   - The tokenizer queries the learned merge rules in priority order. Merge Rule 1 ('u','g') → 'ug' matches immediately! The sequence is rewritten as:

$$[\texttt{'p'}, \texttt{'u'}, \texttt{'g'}, \texttt{'s'}] \xrightarrow{\text{Merge 1}} [\texttt{'p'}, \texttt{'ug'}, \texttt{'s'}].$$

- Merge Rule 2 (`'h','ug'`) → `'hug'` is queried next. It does not match because the preceding symbol is `'p'`, not `'h'`.
- No further merge rules match adjacent symbols.

3. The final emitted token sequence is:

$$\texttt{encode("pugs")} = \texttt{['p', 'ug', 's']}. \tag{1.89}$$

The Out-of-Vocabulary barrier is overcome: the model represents the novel word via known morphological primitives—the initial consonant `'p'`, the shared stem `'ug'`, and the plural suffix `'s'`.

Figure 1.7 visualizes both the bottom-up hierarchical merge tree constructed during training and the top-down decomposition of unseen words at inference time.

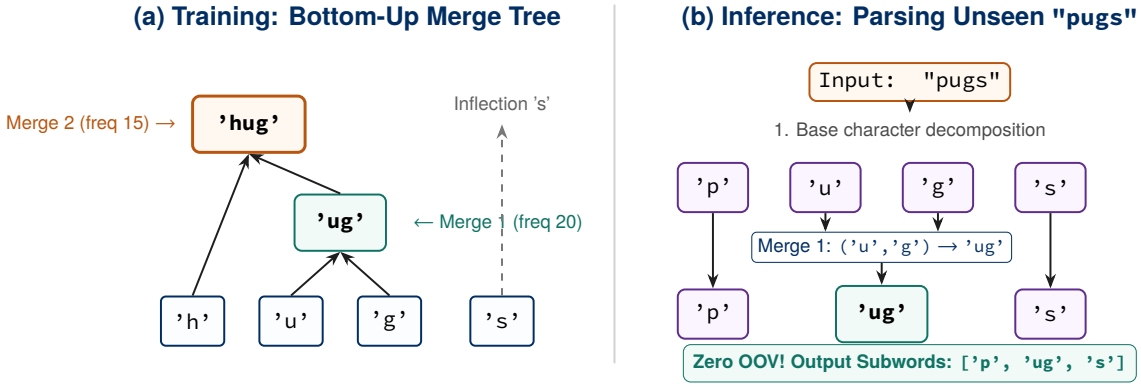


Figure 1.7: Byte Pair Encoding (BPE) mechanics. (a) During training, adjacent high-frequency symbol pairs are greedily merged into subwords. (b) At inference time, unseen words (e.g., `"pugs"`) are parsed by applying learned merge rules in priority order, resolving morphology without out-of-vocabulary collapse.

1.11.4 Byte-level BPE: the 256 UTF-8 byte foundation and elimination of unknown tokens

Despite the success of character-level BPE, applying it directly to Unicode strings faces a severe scaling barrier: the Unicode standard contains over 150,000 unique characters spanning hundreds of languages and symbol sets. Initializing $\mathcal{V}_0$ with every conceivable character creates an unmanageably large base alphabet, yet still fails when a user inputs rare historical glyphs or newly released emojis.

The 256 UTF-8 base byte formulation

In GPT-2, this dilemma was solved in [**?**] by introducing **Byte-Level Byte Pair Encoding (BBPE)**. Instead of characters, the base alphabet $\mathcal{V}_0$ is initialized strictly with the **256 distinct byte values** ($0x00$ through $0xFF$) of the UTF-8 encoding standard:

$$\mathcal{V}_0 = \{0x00, 0x01, \ldots, 0xFF\}, \quad |\mathcal{V}_0| = 256. \tag{1.90}$$

Theorem 1.4: *Total elimination of unknown tokens in byte-level BPE*

Under Byte-Level BPE with base alphabet $\mathcal{V}_0 = \{0x00, \ldots, 0xFF\}$, the probability of an Out-of-Vocabulary (OOV) token is strictly zero:

$$\mathbb{P}(w_{\text{oov}}) = 0 \quad \forall S \in \Sigma^*. \tag{1.91}$$

By definition of the Unicode transformation format (UTF-8), any arbitrary character or sequence of symbols in the Unicode code space ($U+0000$ to $U+10FFFF$) encodes deterministically into a sequence of between 1 and 4 raw bytes in $\{0x00, \ldots, 0xFF\}$. Because the base alphabet $\mathcal{V}_0$ contains all 256 possible 8-bit byte values, every byte sequence can be parsed. If an input string contains an unseen compound word, a programming syntax construct, a severe typo, or an unknown emoji, the tokenizer attempts to apply merged subword rules; if no higher-level merge rule applies, the tokenizer simply falls back to emitting individual byte tokens from $\mathcal{V}_0$. Because no input byte can ever fall outside $\{0x00, \ldots, 0xFF\}$, no token can ever be unmapped. The token `<|unk|>` is completely eliminated. ■

Lossless whitespace preservation

Standard natural language processing pipelines strip whitespace or split on spaces, making exact detokenization ambiguous. In Byte-Level BPE, whitespaces are treated as regular characters/bytes. In GPT-2 and Hugging Face tokenizers, a leading whitespace is mapped to a special character (e.g., ' `extvisiblespace`' or byte $0x20$), binding the space directly to the succeeding word:

$$\texttt{"Hello world"} \overset{\text{BBPE}}{\Longrightarrow} \texttt{['Hello', ' extvisiblespaceworld']}$$

This guarantees that tokenization is **strictly lossless and fully invertible**:

$$\texttt{decode}(\texttt{encode}(S)) \equiv S \quad \forall S \in \Sigma^*. \tag{1.92}$$

Production implementations: OpenAI's tiktoken

In modern production LLM engineering, tokenization must process terabytes of training data without creating CPU bottlenecks. OpenAI's open-source library `tiktoken` implements byte-level BPE in pure Rust, executing tokenization orders of magnitude faster than Python regular expressions:

- **GPT-2 Tokenizer** (`gpt2`): Vocabulary size $V = 50{,}257$ (composed of 256 base byte tokens +50,000 learned BPE merges +1 special `<|endoftext|>` token).
- **GPT-4 Tokenizer** (`cl100k_base`): Vocabulary size $V = 100{,}277$. Expanding vocabulary from 50k to 100k reduces the total token count of natural language and code by $15 - 20\%$, directly reducing sequence length L, decreasing GPU memory footprint, and slashing API inference costs.

```python
import tiktoken

# Instantiate the GPT-2 tokenizer (vocab size 50,257)
tokenizer = tiktoken.get_encoding("gpt2")

text = "Hello, do you like tea? <|endoftext|> In the sunlit terraces of
    someunknownPlace."
# Disallow special tokens check or explicitly allow specific ones
integers = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
```

```
print("Token IDs:", integers)

# Zero-shot handling of unknown words:
# 'someunknownPlace' is decomposed into subwords: ['some', 'unknown', 'Place']
for token_id in integers:
    print(f"{token_id:>5} -> {tokenizer.decode([token_id])!r}")

# Lossless roundtrip reconstruction
decoded_text = tokenizer.decode(integers)
assert decoded_text == text, "Decoded text must match original string identically!"
print("Lossless decoding confirmed!")
```

Listing 1.2: Byte-level BPE tokenization using OpenAI's `tiktoken` library [**?**].

Having established how Byte-Level BPE eliminates the out-of-vocabulary barrier and deterministically encodes raw text streams into discrete integer token IDs $\mathbf{T} = [t_0, t_1, \ldots, t_{L-1}] \in \mathbb{Z}^L$, we now confront the final representational bridge: how do discrete numerical indices enter the continuous, differentiable computational graph of modern neural networks? In the following subsections, we first establish the high-level architecture of the language model input pipeline, and then embark on an exhaustive investigation into positional representations.

1.11.5 From token IDs to composite inputs: high-level architecture of the foundation model input pipeline

Having transformed raw text into sequences of discrete token IDs $\mathbf{T} \in \mathbb{Z}^L$ (batched into mini-batch integer tensors $\mathbf{X} \in \mathbb{Z}^{B \times L}$), modern foundation models execute a unified input preparation pipeline that maps discrete token indices into continuous, differentiable vector representations equipped with sequential order prior to Transformer ingestion.

Delimiter and architectural control tokens

Before projecting indices into continuous embedding space, language models inject dedicated **special control tokens** into the sequence to regulate generation boundaries and downstream task execution:

- **Sequence demarcation tokens**:
  - ⟨BOS⟩ (*Beginning of Sequence*): Prepended to every sequence to prime autoregressive decoders, signaling the onset of generation.
  - ⟨EOS⟩ (*End of Sequence*): Appended to target text; during inference, emitting this token instructs the generation loop to terminate.
- **Architectural and task-specific delimiter tokens**:
  - [CLS] (*Classification token*): Introduced in bidirectional encoder architectures such as BERT. Prepend-ed at sequence index 0, its final contextual representation aggregates whole-sequence semantics for sentence classification or regression tasks.
  - [SEP] (*Separator token*): Delimits distinct segments within a single combined input sequence (for example, demarcating the boundary between a Question and an Answer, or between a code docstring and its method body), enabling the model to learn segment-specific interaction patterns.

The token embedding lookup layer

In PyTorch, continuous token representations are instantiated via an embedding matrix:

```
token_embedding_layer = torch.nn.Embedding(num_embeddings=vocab_size, embedding_dim=d)
```

Passing batch tensor $\mathbf{X} \in \mathbb{Z}^{B \times L}$ retrieves the corresponding continuous embedding vectors:

$$\mathbf{E}_{\text{tok}} = \texttt{Embedding}(\mathbf{X}) \in \mathbb{R}^{B \times L \times d}, \tag{1.93}$$

where slice $(\mathbf{E}_{\text{tok}})_{b,t,:} = \mathbf{w}_{X_{b,t}} \in \mathbb{R}^d$ is the d-dimensional embedding vector for the token at position t in batch item b.

Composite Transformer input synthesis

To endow the model with sequential awareness, the input pipeline combines the lexical token representation $\mathbf{E}_{\text{tok}}$ and a dedicated positional tensor $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{1 \times L \times d}$ via **elementwise addition** rather than concatenation:

$$\mathbf{X}_{\text{in}} = \mathbf{E}_{\text{tok}} + \mathbf{E}_{\text{pos}} \in \mathbb{R}^{B \times L \times d}, \tag{1.94}$$

where for each batch sample $b \in \{0, \ldots, B-1\}$ and temporal coordinate $pos \in \{0, \ldots, L-1\}$:

$$(\mathbf{X}_{\text{in}})_{b,pos,:} = \mathbf{w}_{X_{b,pos}} + \mathbf{p}_{pos} \in \mathbb{R}^d. \tag{1.95}$$

Listing 1.4 demonstrates how this composite synthesis is executed concretely in PyTorch using tensor broadcasting.

```
import torch

vocab_size = 50257     # GPT-2 BPE vocabulary size
output_dim = 256       # Latent embedding dimension d
context_length = 4     # Sequence length L

# 1. Allocate trainable token embedding layer
token_embedding_layer = torch.nn.Embedding(vocab_size, output_dim)

# 2. Allocate positional embedding layer (e.g., learned or precomputed sinusoidal)
pos_embedding_layer = torch.nn.Embedding(context_length, output_dim)

# Example batch of token IDs: [batch_size=2, context_length=4]
inputs = torch.tensor([[40, 367, 2885, 1464],
                       [1807, 3619, 402, 271]])

# Extract token embeddings: [2, 4, 256]
token_embeddings = token_embedding_layer(inputs)

# Generate position indices: tensor([0, 1, 2, 3]) -> [4, 256]
pos_indices = torch.arange(context_length)
pos_embeddings = pos_embedding_layer(pos_indices)

# 3. Additive composite synthesis with PyTorch tensor broadcasting
# token_embeddings: [2, 4, 256] + pos_embeddings: [4, 256] -> [2, 4, 256]
input_embeddings = token_embeddings + pos_embeddings

print("Token Embeddings Shape:", token_embeddings.shape)  # torch.Size([2, 4, 256])
print("Pos Embeddings Shape:  ", pos_embeddings.shape)    # torch.Size([4, 256])
print("Final Composite Shape: ", input_embeddings.shape)  # torch.Size([2, 4, 256])
```

Listing 1.3: PyTorch synthesis of token embeddings and positional embeddings [**?**].

While Equation (1.97) and Listing 1.4 summarize the high-level computational contract of the LM input pipeline, they raise foundational questions: *Why is the positional tensor* $\mathbf{E}_{pos}$ *strictly mandatory? Why do naive indexing schemes fail catastrophically? And what mathematical principles govern its construction?* We now dedicate the next subsection to an exhaustive investigation into positional representations.

1.11.6 Positional representations: intuition, mathematical formulation, and implementations

Permutation invariance dilemma in self-attention

In classical Recurrent Neural Networks (RNNs), sequential order is implicitly baked into the architecture: the hidden state transitions recurrently via $\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$, guaranteeing that token $\mathbf{x}_t$ is processed strictly after token $\mathbf{x}_{t-1}$. In Convolutional Neural Networks (CNNs), sequential order is preserved through localized spatial receptive fields.

In contrast, the cornerstone mathematical operator of the Transformer—**Self-Attention**—is strictly **permutation-equivariant**:

Theorem 1.5: *Permutation equivariance of self-attention*

Let $\mathbf{X} \in \mathbb{R}^{L\times d}$ be an input sequence of L vectors, and let $\mathbf{P} \in \{0,1\}^{L\times L}$ be an arbitrary permutation matrix (a square matrix having exactly one entry 1 per row and column, satisfying $\mathbf{P}^\top = \mathbf{P}^{-1}$). Under standard dot-product self-attention:

$$\texttt{Attention}(\mathbf{X}) = \text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q(\mathbf{X}\mathbf{W}_K)^\top}{\sqrt{d_k}}\right)\mathbf{X}\mathbf{W}_V, \tag{1.96}$$

the operator is equivariant to input permutation:

$$\texttt{Attention}(\mathbf{P}\mathbf{X}) = \mathbf{P}\,\texttt{Attention}(\mathbf{X}). \tag{1.97}$$

Proof. Let $\mathbf{X}' = \mathbf{P}\mathbf{X}$. The query and key projections evaluate to $\mathbf{Q}' = \mathbf{P}\mathbf{X}\mathbf{W}_Q$ and $\mathbf{K}' = \mathbf{P}\mathbf{X}\mathbf{W}_K$. The unscaled attention score matrix becomes:

$$\mathbf{Q}'(\mathbf{K}')^\top = (\mathbf{P}\mathbf{X}\mathbf{W}_Q)(\mathbf{P}\mathbf{X}\mathbf{W}_K)^\top = \mathbf{P}(\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top)\mathbf{P}^\top. \tag{1.98}$$

Because the Softmax function is evaluated independently along each row, applying a permutation matrix $\mathbf{P}$ on the left permutes the rows, while $\mathbf{P}^\top$ on the right permutes the columns:

$$\text{softmax}\left(\frac{\mathbf{P}(\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top)\mathbf{P}^\top}{\sqrt{d_k}}\right) = \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)\mathbf{P}^\top. \tag{1.99}$$

Finally, multiplying by the value projection $\mathbf{V}' = \mathbf{P}\mathbf{X}\mathbf{W}_V$:

$$\begin{aligned}\texttt{Attention}(\mathbf{P}\mathbf{X}) &= \left[\mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)\mathbf{P}^\top\right](\mathbf{P}\mathbf{X}\mathbf{W}_V)\\ &= \mathbf{P}\,\text{softmax}\left(\frac{\mathbf{X}\mathbf{W}_Q\mathbf{W}_K^\top\mathbf{X}^\top}{\sqrt{d_k}}\right)(\mathbf{P}^\top\mathbf{P})\mathbf{X}\mathbf{W}_V\\ &= \mathbf{P}\,\texttt{Attention}(\mathbf{X}),\end{aligned} \tag{1.100}$$

where the identity $\mathbf{P}^\top\mathbf{P} = \mathbf{I}$ collapses the internal permutation. ■ □

Theorem 1.5 establishes that if we permute the input tokens, the output vectors undergo the exact same permutation with *identical values*. The attention mechanism treats the input sequence as an **unordered multiset** (a "bag of vectors") with zero intrinsic knowledge of temporal order!

To understand the catastrophic consequence of this property, consider two contrasting sentence pairs:

$$\begin{aligned} \textbf{Sentence 1}\text{: } \textit{"I ate an apple"} &\implies \{\mathbf{e}_{\text{I}}, \mathbf{e}_{\text{ate}}, \mathbf{e}_{\text{an}}, \mathbf{e}_{\text{apple}}\} \\ \textbf{Sentence 2}\text{: } \textit{"An apple, I ate"} &\implies \{\mathbf{e}_{\text{an}}, \mathbf{e}_{\text{apple}}, \mathbf{e}_{\text{I}}, \mathbf{e}_{\text{ate}}\} \end{aligned}$$

Or similarly:

$$\begin{aligned} \textbf{Sentence A}\text{: } \textit{"The dog bit the man"} &\implies \{\mathbf{e}_{\text{The}}, \mathbf{e}_{\text{dog}}, \mathbf{e}_{\text{bit}}, \mathbf{e}_{\text{the}}, \mathbf{e}_{\text{man}}\} \\ \textbf{Sentence B}\text{: } \textit{"The man bit the dog"} &\implies \{\mathbf{e}_{\text{The}}, \mathbf{e}_{\text{man}}, \mathbf{e}_{\text{bit}}, \mathbf{e}_{\text{the}}, \mathbf{e}_{\text{dog}}\} \end{aligned}$$

In both pairs, the sentences express fundamentally different or opposite semantic realities. Yet, because each pair comprises the exact same multiset of token embeddings, an unaugmented Transformer evaluates identical attention weights. To the model, subject, verb, and object become entirely conflated. Sequential order must therefore be injected into the representations explicitly.

Why naive positional indexing fails

Before arriving at the modern formulation, it is vital to understand why naive indexing schemes break down:

1. **Naive Integer Addition ($pos \in \{0, 1, 2, \ldots, L-1\}$)**: If we add scalar indices pos to token embeddings, early tokens receive tiny perturbations (+0, +1), while token 2048 receives +2048. Because embedding vectors are initialized with mean 0 and unit variance, large coordinates overwhelm the semantic representations completely, and models fail to generalize to sequence lengths beyond training.

2. **Normalized Coordinates ($pos/L \in [0, 1]$)**: Normalizing positions by sequence length bounds values within $[0, 1]$. However, the physical meaning of a step size becomes variable: in a 10-token sentence, an offset of $\Delta = 0.1$ corresponds to an adjacent word, whereas in a 1000-token document, $\Delta = 0.1$ spans 100 words. The model cannot learn consistent relative distances.

3. **Coordinate Concatenation ($[\mathbf{w}_t; \mathbf{p}_t]$)**: Concatenating positional coordinates expands embedding dimensionality to $d + d_{\text{pos}}$, increasing parameter counts across all subsequent projection matrices ($\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$) and preventing seamless parameter sharing.

The geometric clock intuition for positional encoding

How can we construct an encoding that remains strictly bounded within $[-1, 1]$, assigns a unique signature to every position, and preserves consistent geometric notions of distance?
From binary counting to continuous multi-scale clocks.
To build geometric intuition, consider how a digital computer represents numbers using bounded bits $\{0, 1\}$. In a 3-bit binary counter:

- The lowest bit (Bit 0) alternates every single step: $0, 1, 0, 1, 0, 1 \ldots$ (high-frequency wave).
- The middle bit (Bit 1) alternates every two steps: $0, 0, 1, 1, 0, 0 \ldots$ (medium-frequency wave).
- The highest bit (Bit 2) alternates every four steps: $0, 0, 0, 0, 1, 1 \ldots$ (low-frequency wave).

Every integer receives an unambiguous signature, even though individual bits are bounded in $\{0, 1\}$. However, discrete step functions are non-differentiable. Sinusoidal positional encoding operates as the smooth, continuous, differentiable generalization of binary counting.

Rather than binary square waves, we assign continuous trigonometric waves oscillating at geometrically decreasing frequencies across embedding channels:

- **Fast Clocks (Seconds Hand)**: Oscillate rapidly across low dimension indices, capturing fine-grained local syntax and adjacent word order.
- **Medium Clocks (Minutes Hand)**: Oscillate at intermediate frequencies, capturing phrase and clause boundaries.
- **Slow Clocks (Hours Hand)**: Oscillate extremely slowly across high dimension indices, capturing document-level flow and paragraphs.

Why can we not simply use sine waves with varying frequencies? Consider an analog clock where you can only observe the vertical height of the hand ($Y = \sin\theta$). Because $\sin(\theta) = \sin(\pi - \theta)$, the hand is at the exact same vertical height at 2:00 and 10:00! Furthermore, $\sin(0) = \sin(\pi) = 0$. A single 1D wave suffers from severe directional ambiguity (aliasing).

To establish an unambiguous position, we need both coordinates of the clock face:

$$(X, Y) = (\cos\theta, \sin\theta). \tag{1.101}$$

Together, $(\sin\theta, \cos\theta)$ traces a point along the unit circle S^1. On a 2D circle, every angular orientation $\theta \in [0, 2\pi)$ has an unambiguous coordinate! This is why Transformer positional encodings alternate sin and cos across dimension pairs.

Figure **??** illustrates this multi-frequency clock mechanism and demonstrates how pairing sine and cosine eliminates 1D wave aliasing.

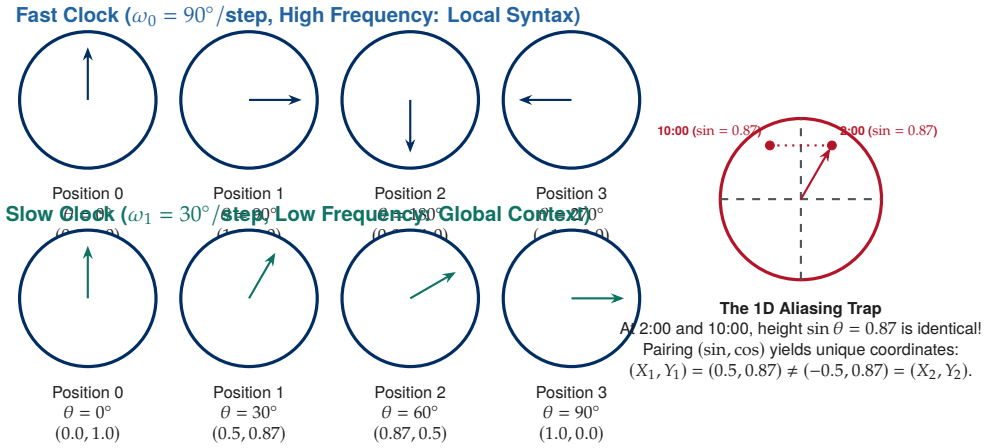


Figure 1.8: The geometric clock intuition behind positional encodings. Each dimension pair acts as an analog clock hand rotating at an angular frequency ω_i. Fast clocks disambiguate immediate neighbors (local syntax), while slow clocks preserve long-range sequence context. Pairing sin and cos prevents 1D wave aliasing by establishing a unique 2D coordinate on the unit circle S^1.

Taxonomy of positional representations

Modern architectures implement positional awareness through three primary paradigms:

1. **Fixed Sinusoidal Positional Encodings [?]**: Fully deterministic, parameter-free functions computed from geometric progressions of trigonometric frequencies.

2. **Learned Absolute Positional Embeddings [? ? ?]**: Trainable parameter weights stored in an embedding matrix $\mathbf{W}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d}$ optimized via backpropagation (used in GPT-2, GPT-3, and BERT).
3. **Relative and Rotary Positional Embeddings (RoPE / ALiBi) [? ? ?]**: Encodings that inject positional information directly into attention scores as a function of relative distance $|i - j|$. Notably, Rotary Position Embedding (RoPE) [**?**] directly generalizes the 2D rotation matrix property of sinusoidal encodings into query and key representations.

Sinusoidal positional encodings mathematical derivation

In the seminal architecture introduced in [**?**], positional coordinates are generated deterministically:

Definition 1.6: *Sinusoidal positional encoding*

For a token at temporal sequence coordinate $pos \in \{0, 1, \ldots, L-1\}$ and channel index $i \in \{0, 1, \ldots, \frac{d}{2} - 1\}$, the components of the positional vector $\mathbf{p}_{pos} = \mathsf{PE}_{(pos,:)} \in \mathbb{R}^d$ are defined by:

$$\mathsf{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \tag{1.102}$$

$$\mathsf{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right). \tag{1.103}$$

Let us examine every term in this formulation:

- pos: The integer position of the token within the sequence $(0, 1, 2, \ldots, L-1)$.
- i: The frequency channel index $(0 \leq i < d/2)$.
- $2i$ and $2i + 1$: Even indices receive sine values, odd indices receive cosine values.
- Angular frequency ω_i: Defined as $\omega_i = \frac{1}{10000^{2i/d}} = 10000^{-2i/d}$.
- Wavelength λ_i: The wavelength $\lambda_i = \frac{2\pi}{\omega_i} = 2\pi \cdot 10000^{2i/d}$ forms a geometric progression:

$$\text{At } i = 0: \quad \lambda_0 = 2\pi \cdot 10000^0 = 2\pi \approx 6.28 \text{ tokens}, \tag{1.104}$$

$$\text{At } i = \frac{d}{2} - 1: \quad \lambda_{\max} \approx 2\pi \cdot 10000^1 \approx 62{,}832 \text{ tokens}. \tag{1.105}$$

- **Why the base 10000?** As highlighted by the professor, 10,000 guarantees that within typical context lengths (up to $\approx$ 60,000 tokens), the slowest frequency does not complete a single cycle. This guarantees that every position vector in the operational horizon is globally unique.

Sinusoidal positional encodings satisfy three pivotal theoretical properties:

1. **Positional uniqueness**: Because each channel i oscillates at a distinct frequency ω_i, no two positional vectors are identical. The vectors remain distinct for at least $2\pi \cdot 10000 \approx 62{,}832$ positions before periodic recurrence can occur.
2. **The linear relative shift property**: For any fixed spatial offset $k \in \mathbb{Z}$, the positional vector at position $pos + k$ can be computed as a direct linear transformation of the vector at position pos:

Theorem 1.6: *Linear relative transformation property*

For any frequency $\omega_i = 10000^{-2i/d}$ and spatial offset $k \in \mathbb{Z}$, there exists an orthogonal rotation matrix $\mathbf{M}_{i,k} \in \mathbb{R}^{2\times 2}$ independent of pos such that:

$$\begin{bmatrix} \mathsf{PE}_{(pos+k,2i)} \\ \mathsf{PE}_{(pos+k,2i+1)} \end{bmatrix} = \begin{bmatrix} \cos(\omega_i k) & \sin(\omega_i k) \\ -\sin(\omega_i k) & \cos(\omega_i k) \end{bmatrix} \begin{bmatrix} \mathsf{PE}_{(pos,2i)} \\ \mathsf{PE}_{(pos,2i+1)} \end{bmatrix}. \tag{1.106}$$

Proof. Applying the standard trigonometric angle addition identities:

$$\sin(\omega_i(pos+k)) = \sin(\omega_i pos)\cos(\omega_i k) + \cos(\omega_i pos)\sin(\omega_i k), \tag{1.107}$$
$$\cos(\omega_i(pos+k)) = \cos(\omega_i pos)\cos(\omega_i k) - \sin(\omega_i pos)\sin(\omega_i k). \tag{1.108}$$

Expressing this system in matrix notation yields Equation (1.104). Because the rotation matrix $\mathbf{M}_{i,k}$ depends strictly on offset k and is independent of absolute position pos, linear attention projection layers ($\mathbf{W}_Q, \mathbf{W}_K$) can easily learn to attend to relative positions. ■ □

3. **Metric similarity and distance decay**: In self-attention, query and key vectors interact via inner products. The dot product between the 2D positional components of two tokens at positions p_1 and p_2 evaluates to:

$$\mathbf{p}_{p_1}^{(i)} \cdot \mathbf{p}_{p_2}^{(i)} = \sin(\omega_i p_1)\sin(\omega_i p_2) + \cos(\omega_i p_1)\cos(\omega_i p_2) = \cos(\omega_i(p_1 - p_2)). \tag{1.109}$$

The dot product depends **strictly on the relative distance** $\Delta p = p_1 - p_2$:

- When $p_1 = p_2$, $\cos(0) = 1$ (maximum correlation along the diagonal).
- As $|p_1 - p_2|$ increases, the similarity smoothly decays.

Computing the complete pairwise cosine similarity matrix across 500 positions demonstrates this property empirically:

- The 2D Gram matrix exhibits a bright, luminous main diagonal (cos = 1).
- A horizontal slice along row $pos = 100$ reveals a sharp symmetric peak at index 100, followed by a smooth, monotonic decay as distance $|pos - 100|$ expands.

Numerical calculation of sinusoidal coordinates

To illustrate this formulation concretely, consider a canonical numerical configuration: sequence position $pos = 1$ (the second token, under 0-indexed coordinates) within a standard latent embedding space of dimension $d = 768$:

1. **Lowest channel pair ($i = 0$, high-frequency regime)**:

$$2i = 0 \implies \mathsf{PE}_{(1,0)} = \sin\left(\frac{1}{10000^{0/768}}\right) = \sin(1) \approx 0.8415, \tag{1.110}$$
$$2i + 1 = 1 \implies \mathsf{PE}_{(1,1)} = \cos\left(\frac{1}{10000^{0/768}}\right) = \cos(1) \approx 0.5403. \tag{1.111}$$

2. **Subsequent even channel ($i = 2$, channel index $2i = 4$)**:

$$2i = 4 \implies \mathsf{PE}_{(1,4)} = \sin\left(\frac{1}{10000^{4/768}}\right) = \sin\left(\frac{1}{10000^{0.005208}}\right)$$
$$\approx \sin\left(\frac{1}{1.0490}\right) = \sin(0.9533) \approx 0.8153. \tag{1.112}$$

These exact values (0.8415, 0.5403, and 0.8153) illustrate how trigonometric arguments shift smoothly across embedding indices, providing each positional coordinate with an unambiguous, deterministic signature.

Learned positional embeddings (GPT convention)

An alternative approach, adopted by architectures such as GPT-2 and GPT-3 [**?** **?**], treats positional encodings as trainable parameter weights rather than fixed sinusoidal functions:

$$\mathbf{W}_{\text{pos}} \in \mathbb{R}^{L_{\max} \times d}. \tag{1.113}$$

Each integer coordinate $p \in \{0, 1, \ldots, L_{\max} - 1\}$ indexes a row vector $\mathbf{p}_p = (\mathbf{W}_{\text{pos}})_{p,:} \in \mathbb{R}^d$ optimized via gradient descent alongside token embeddings.

Table 1.3: Architectural comparison: sinusoidal positional encodings vs. learned positional embeddings.

Property	Sinusoidal encodings [?]	Learned embeddings [?]
Parameters	0 trainable weights (fully deterministic)	$L_{\max} \times d$ trainable parameters
Extrapolation	Can theoretically extrapolate to sequence lengths $> L_{\max}$	Strictly confined to context length $L \leq L_{\max}$
Relative shift	Exact orthogonal rotation $\mathbf{M}_{i,k}$ provable analytically	Must be approximated empirically from data
Empirical similarity	Pure smooth monotonic distance decay around diagonal	Smooth decay near diagonal with data-driven periodic peaks
Empirical performance	Comparable translation BLEU in [**?**]	Standard in modern autoregressive decoders

Computing the cosine similarity matrix of learned GPT-2 embeddings reveals high values along the main diagonal, accompanied by periodic secondary peaks at larger distances. These secondary peaks reflect statistical regularities in the training corpus (such as paragraph transitions, section breaks, or dialogue conventions).

Walkthrough: disambiguating opposite sequential orders

To crystallize all of these principles into an indelible mental picture, let us trace how a mini 2-clock system disambiguates our contrasting sentences:

Sentence 1: *"The dog bit the man."* vs. **Sentence 2**: *"The man bit the dog."*

Let $d = 4$ (consisting of 2 clocks):

- **Clock 1 (Fast Clock, $\omega_0 = 90°$/step)**: Rotates 90° clockwise per step.
- **Clock 2 (Slow Clock, $\omega_1 = 30°$/step)**: Rotates 30° clockwise per step.

Evaluating the positional vectors $\mathbf{PE}(pos) = [\sin(\omega_0 pos), \cos(\omega_0 pos), \sin(\omega_1 pos), \cos(\omega_1 pos)]$ yields:

Now consider the additive inputs:

- In **Sentence 1**: $\mathbf{x}_{\text{dog}} = \mathbf{E}_{\text{dog}} + \mathbf{p}_1$, $\mathbf{x}_{\text{bit}} = \mathbf{E}_{\text{bit}} + \mathbf{p}_2$, $\mathbf{x}_{\text{man}} = \mathbf{E}_{\text{man}} + \mathbf{p}_4$.
- In **Sentence 2**: $\mathbf{x}_{\text{man}} = \mathbf{E}_{\text{man}} + \mathbf{p}_1$, $\mathbf{x}_{\text{bit}} = \mathbf{E}_{\text{bit}} + \mathbf{p}_2$, $\mathbf{x}_{\text{dog}} = \mathbf{E}_{\text{dog}} + \mathbf{p}_4$.

Table 1.4: Exact numerical positional vectors for positions 0 through 4 under the 2-clock system.

Position (pos)	**Clock 1 ($\omega_0 = 90° \cdot pos$)**	**Clock 2 ($\omega_1 = 30° \cdot pos$)**	**Positional Vector $\mathbf{p}_{pos}$**
0	$\theta = 0° \implies [0.0, 1.0]$	$\theta = 0° \implies [0.0, 1.0]$	$[0.0, 1.0, 0.0, 1.0]$
1	$\theta = 90° \implies [1.0, 0.0]$	$\theta = 30° \implies [0.5, 0.87]$	$[1.0, 0.0, 0.5, 0.87]$
2	$\theta = 180° \implies [0.0, -1.0]$	$\theta = 60° \implies [0.87, 0.5]$	$[0.0, -1.0, 0.87, 0.5]$
3	$\theta = 270° \implies [-1.0, 0.0]$	$\theta = 90° \implies [1.0, 0.0]$	$[-1.0, 0.0, 1.0, 0.0]$
4	$\theta = 360° \implies [0.0, 1.0]$	$\theta = 120° \implies [0.87, -0.5]$	$[0.0, 1.0, 0.87, -0.5]$

In English syntax, the grammatical subject of the active verb "bit" resides 1 step prior (relative offset $k = -1$). At position $pos = 2$, Clock 1 points South (180°, vector $[0.0, -1.0]$). To identify the doer of the action, an attention head querying offset $k = -1$ rotates 90° counterclockwise:

$$180° - 90° = 90° \implies [1.0, 0.0]. \tag{1.114}$$

The attention query searches for the token whose Clock 1 points East ($[1.0, 0.0]$):

- In **Sentence 1**, the token at position 1 with East-pointing clock is **"dog"**. The attention mechanism routes information from "dog" to "bit", resolving the dog as the subject.
- In **Sentence 2**, the token at position 1 with East-pointing clock is **"man"**. The attention mechanism routes information from "man" to "bit", resolving the man as the subject.

The ambiguity is completely resolved: through sinusoidal clock rotations, the Transformer effortlessly deciphers who bit whom!